

Health Monitoring System with Real Time Alerts

A Project report submitted in partial fulfillment of the requirements For

the award of the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

By

Md Manzar Nizam

(Regd. No.: 321136410079)

Ragathi Sreelekha

(Regd. No.: 321136410101)

Koyilada Keerti

(Regd. No.: 321136410068)

Nakka Shiva Kushal Vikram

(Regd. No.: 321136410086)

Under the esteemed guidance of

Ms. Y. Sree Chithra

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Dr. LANKAPALLI BULLAYYA COLLEGE OF ENGINEERING

(Permanently Affiliated to Andhra University, Approved by AICTE, New Delhi)

Accredited by NAAC with B++ Grade, Included under section 2(f) of the UGC Act

1956 New Resapuvanipalem, Visakhapatnam, Andhra Pradesh-530013

Website: www.lbce.edu.in

Year of Submission 2025



Dr. LANKAPALLI BULLAYYA COLLEGE OF ENGINEERING

(Permanently Affiliated to Andhra University, Approved by AICTE, New Delhi)

Accredited by NAAC with B++ Grade, Included under section 2(f) of the UGC Act 1956

New Resapuvanipalem, Visakhapatnam, Andhra Pradesh-530013

Website: www.lbce.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that **Mr. Md Manzar Nizam, Miss. Ragathi Sreelekha, Miss. Koyilada Keerti, Mr. Nakka Shiva Kushal Vikram** bearing register numbers **321136410079, 321136410101, 321136410068, 321136410086** respectively, the students of final year B.Tech in Computer Science Engineering, has carried out the project work titled “**Health Monitoring System with Real Time Alerts**” at Dr. Lankapalli Bullayya College of Engineering, Visakhapatnam during the academic year **2024-25**.

Project Supervisor

Ms.Y.Sree Chithra

Assistant.Professor

Dept. of Computer Science Engineering

Head of the Department

Dr.D.Madhavi

Professor

Dept. of Computer Science Engineering

EXTERNAL EXAMINER

ABSTRACT

The Health Monitoring System with Real Time Alerts is a real-time health tracking solution that integrates machine learning and AI to enhance user safety and emergency response. The system continuously monitors key health parameters such as blood pressure, heart rate variability (HRV), blood sugar levels, and oxygen saturation (SpO2). A fall prediction mechanism assesses potential risks and triggers immediate alerts to caregivers or medical professionals.

An AI-powered chatbot provides personalized health recommendations, while automated SMS and call alerts ensure rapid communication in critical situations. The system employs machine learning models, including SVM, Logistic Regression, KNN, Naïve Bayes, and Random Forest, to detect anomalies and predict potential health risks.

Designed with a Streamlit-based web interface, the system offers an intuitive user experience. In CSV files it stores user records for long-term analysis, enabling proactive healthcare measures. By integrating real-time monitoring, predictive analytics, and emergency alert mechanisms, this project delivers an efficient and accessible solution for modern healthcare needs.

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to **Prof. D. Deepak Chowdary**, Principal, Dr. L. Bullayya College of Engineering, Visakhapatnam, for giving us this opportunity to do this project.

We would like to thank **Prof. D. Madhavi**, HOD, Dept. of C.S.E, Dr.L.Bullayya College of Engineering, Visakhapatnam, for her guidance and assistance in producing this work.

We are deeply indebted to head of the research cluster “**Computational Intelligence**” **Dr. D. RajaKishor** Professor, Department of Computer Science Engineering, and our project guide **Ms. Y. Sree Chithra** Assistant. Professor, Department of Computer Science Engineering, Dr. L. Bullayya College of Engineering, Visakhapatnam, for guiding us throughout the project inspite of their busy schedule.

Apart from the efforts of me, the success of this project depends largely on the encouragement of other Faculty of C.S.E, Dr. L. Bullayya College of Engineering Visakhapatnam. We take this opportunity to express our gratitude to the entire faculty who has been instrumental in the successful completion of this project.

Also deserving of thanks are my family and friends, for their support for their confidence in my achievements.

Md Manzar Nizam

Ragathi Sreelekha

Koyilada Keerti

Nakka Shiva Kushal Vikram

DECLARATION

This is to declare that the project work entitled "**Health Monitoring System with Real Time Alerts**", is a bonafide work done by us, under the research cluster **Computational Intelligence** in Department of CSE, **Dr. L. Bullayya College Of Engineering**. This project report is being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science Engineering during the academic year 2024-25. This project possesses originality as it is not extracted from any source and it has not been submitted to any other institution or university.

(Md Manzar Nizam)
Reg.No:321136410079

(Ragathi Sreelekha)
Reg.No:321136410101

(Koyilada Keerti)
Reg.No:321136410068

(Nakka Shiva Kushal Vikram)
Reg.No:321136410086

Visakhapatnam
Date:

LIST OF TABLES

Table no	Name of the Table	Page no
1	SRS	9
2	Scenario	17
3	Performance Comparison	47
4	Software Technologies	48
5	Dataset Characteristics	57
6	Test case	60

LIST OF FIGURES

Figure no	Name of the Figure	Page no
1	Use Case Diagram	19
2	Sequence Diagram	20
3	Class Diagram	28
4	Activity Diagram	30
5	State Chart Diagram	32
6	Component Diagram	35
7	Graph Screenshot	42
8	Output Screenshot	63

TABLE OF CONTENTS

Chapter No	Title	Page No.
	Abstract	iii
	Acknowledgements	iv
	Declaration	v
	List of Tables	vi
	List of Figures	vi
1	Introduction	01
	1.1 Background and Motivation	02
	1.2 Problem Statement	03
2	Requirements Elicitation and Analysis	04
	2.1 Existing System	04
	2.2 Proposed System	05
	2.3 Feasibility Requirements	06
	2.4 System Requirements	08
	2.4.1 Functional Requirements	08
	2.4.2 Non Functional Requirements	10
3	System Design.	14
	3.1 Object Oriented Analysis and Design	14
	3.1.1 Scenarios	15
	3.1.2 Use Case diagram	18
	3.1.3 Sequence diagram	20
	3.1.4 Class diagram	26
	3.1.5 Activity diagram	29
	3.1.6 State Chart diagram	31
	3.1.7 Component diagram	33
	3.2 Database Design	36
4	Implementation Details	37
	4.1 Software Environment	37
	4.1.1 Procedure/Algorithm	37
	4.2 Software Technologies	48
	4.3 Database Details	56
	4.3.1 Tables	57
5	Testing	58
6	Conclusions	61
	References	62
	Appendix	63
	A_ Input/Output Screens	63
	B– Sample Code	71

1. INTRODUCTION

In today's fast-paced world, proactive and accessible healthcare solutions are essential, particularly for vulnerable populations such as the elderly or individuals at risk of falls. The "Health Monitoring System with Real Time Alerts" addresses this need by providing a technology-driven approach to personal health management, focusing on walking stability analysis and fall risk prediction. This system leverages real-time data acquisition, image processing, and artificial intelligence to offer a robust safety net for users, empowering them to monitor their health and prevent potential incidents.

Traditional healthcare models often rely on periodic assessments and reactive interventions, which may fail to detect subtle changes in walking patterns that could indicate an increased risk of falls. This project bridges this gap through a step count-based analysis framework that collects walking data via image uploads (e.g., Samsung Health screenshots processed with Tesseract OCR) and manual data entry. A rule-based stability scoring mechanism evaluates key metrics such as step count, step variability, active hours, and steps per minute to calculate a stability score and predict fall risk in real time, providing immediate feedback to users through a user-friendly interface.

To enhance user experience and accessibility, the system integrates an AI-powered chatbot using the Google Gemini AI API, which delivers personalized health recommendations and answers user queries, making health guidance more interactive and readily available. The system is designed with an intuitive Streamlit-based web interface, featuring visualizations like a stability meter and trend charts, as well as a clear option for image uploads, ensuring ease of use for individuals with varying levels of technical proficiency. Long-term analysis of user data is facilitated through storage in CSV files using Pandas, enabling proactive healthcare measures and personalized insights into walking patterns.

By seamlessly integrating step count-based analysis, a rule-based stability scoring mechanism, real-time fall risk alerts, and AI-driven health assistance, this project delivers an efficient and accessible solution for modern healthcare needs, with the potential for future enhancement using machine learning models like Support Vector Machines (SVM) to further improve fall risk prediction accuracy.

1.1. Background and Motivation

The growing demand for proactive and accessible healthcare solutions has exposed limitations in traditional healthcare systems, which often rely on periodic assessments and delayed interventions. Vulnerable populations, such as the elderly or individuals with mobility challenges, face an elevated risk of falls, a leading cause of injury and hospitalization in these groups. Falls can occur unexpectedly, and without continuous monitoring of walking patterns, the risk of delayed detection and intervention increases significantly. With advancements in artificial intelligence (AI), image processing, and data analytics, it is now possible to analyze walking behavior, predict fall risks, and provide real-time feedback to prevent such incidents. The "Health Monitoring System with Real Time Alerts" leverages these technologies to create a preventive healthcare environment, focusing on walking stability analysis and fall risk prediction for enhanced user safety.

Traditional healthcare approaches often fail to provide continuous monitoring of walking patterns, leaving gaps in detecting subtle changes that may indicate an increased fall risk. This project addresses these gaps through a step count-based analysis framework that collects walking data via image uploads (processed with Tesseract OCR) and manual entry, enabling the system to assess walking stability in real time. A rule-based stability scoring mechanism evaluates key metrics such as step count, step variability, active hours, and steps per minute to predict fall risk, delivering immediate alerts through a user-friendly Streamlit interface. Additionally, an AI-powered chatbot, integrated using the Google Gemini AI API, provides personalized health recommendations, making health guidance more accessible, especially for individuals living alone or with limited access to healthcare facilities.

The motivation behind this project is to reduce the risk of falls, a critical health concern for the elderly and those with mobility issues, by enabling real-time monitoring and predictive analysis of walking patterns. By integrating step count-based analysis, a rule-based stability scoring mechanism, real-time fall risk alerts, and AI-driven health assistance, the system aims to minimize emergency risks, enhance remote health support, and contribute to a technology-driven healthcare ecosystem. Furthermore, the system stores user data in CSV files for long-term analysis, supporting proactive health management and personalized insights. This project seeks to empower users with actionable health information, reduce the burden on caregivers, and pave the way for future enhancements, such as machine learning-based fall risk prediction, to further improve healthcare outcomes.

1.2. Problem Statement

Current healthcare monitoring systems often fall short in proactively detecting fall risks, particularly for vulnerable populations such as the elderly or individuals with mobility issues. Traditional approaches rely on infrequent assessments, manual tracking of walking patterns, and reactive interventions, which are insufficient for identifying subtle changes in walking stability that may indicate an increased risk of falls. The absence of real-time monitoring and automated fall risk prediction mechanisms can lead to delayed detection and intervention, increasing the likelihood of fall-related injuries and subsequent health complications. Additionally, many individuals, especially those living alone or in remote areas, lack immediate access to health guidance, making it challenging to address potential risks promptly.

Manual tracking of walking data is also prone to errors and inconsistencies, leading to inefficient assessments of fall risk. Furthermore, the lack of accessible, user-friendly tools for monitoring walking stability and obtaining health advice limits the ability of users to take proactive measures to prevent falls. This project addresses these challenges by developing the "Health Monitoring System with Real Time Alerts," which:

- Collects walking data through step count-based analysis, using image uploads (processed with Tesseract OCR) and manual data entry to track step counts, active hours, and steps per minute.

- Employs a rule-based stability scoring mechanism to evaluate walking stability and predict fall risk in real time, providing immediate feedback to users.

- Features an AI-powered chatbot, integrated via the Google Gemini AI API, for real-time health recommendations and query resolution.

- Offers a user-friendly Streamlit-based web interface with visualizations (e.g., stability meter, trend charts) and a clear option for image uploads, ensuring seamless interaction.

- Stores and analyzes user walking data in CSV files using Pandas for long-term tracking and proactive health management.

By integrating step count-based analysis, a rule-based stability scoring mechanism, real-time fall risk alerts, and AI-driven health assistance, this system enhances user safety, early detection of fall risks, and accessibility to health guidance. The system aims to reduce the risk of fall-related emergencies, empower users with actionable insights, and pave the way for future enhancements, such as machine learning-based fall risk prediction using models like Support Vector Machines (SVM), to further improve accuracy and effectiveness.

2. REQUIREMENTS ELICITATION AND ANALYSIS

2.1 Existing System

Traditional health monitoring systems primarily rely on manual intervention and periodic assessments, which can lead to delays in detecting fall risks and addressing mobility-related health concerns. These systems often lack real-time monitoring of walking patterns and automated fall risk prediction mechanisms, making them inadequate for preventing fall-related injuries, particularly among vulnerable populations such as the elderly. Some of the major limitations of existing health monitoring systems include:

- **Lack of Real-Time Monitoring:** Walking patterns are typically assessed during scheduled medical appointments or physical therapy sessions, which may result in delayed detection of changes in stability that indicate an increased fall risk.
- **Manual Data Recording:** Walking data, such as step counts or activity levels, is often recorded manually by patients or caregivers, increasing the chances of errors and inconsistencies in tracking long-term patterns.
- **No Predictive Analysis:** Existing systems rarely leverage AI or data analytics to predict fall risks based on historical walking data, relying instead on subjective assessments that may overlook subtle indicators of instability.
- **Limited Fall Risk Detection:** There is no automated mechanism to continuously monitor walking stability and provide immediate feedback on fall risk, leaving users without timely insights to prevent potential incidents.
- **Limited Accessibility:** Individuals in remote areas or those with mobility issues face challenges in accessing regular health assessments and guidance, often lacking the tools to monitor their walking stability independently.

These limitations highlight the need for a more proactive, automated, and accessible system to monitor walking patterns, predict fall risks, and provide real-time health guidance, which the "Health Monitoring System with Real Time Alerts" aims to address through step count-based analysis, a rule-based stability scoring mechanism, and AI-driven health assistance.

2.2 Proposed System

The "Health Monitoring System with Real Time Alerts" addresses the limitations of traditional health monitoring systems by integrating step count-based analysis, real-time fall risk prediction, and AI-driven health assistance. The proposed system offers the following key features:

- **Real-Time Walking Stability Monitoring:** Collects walking data through step count-based analysis, using image uploads (processed with Tesseract OCR to extract step counts and dates from sources like Samsung Health screenshots) and manual data entry to track metrics such as step count, active hours, and steps per minute.
- **Fall Risk Prediction:** Employs a rule-based stability scoring mechanism to evaluate walking stability by analyzing step count, step variability, active hours, and steps per minute, providing real-time fall risk predictions and immediate feedback to users.
- **AI-Powered Chatbot:** Integrates an AI chatbot via the Google Gemini AI API, offering instant health recommendations and guidance based on user queries, enhancing accessibility to health advice.
- **User-Friendly Interface:** Features a Streamlit-based web application with an intuitive interface, including visualizations like a stability meter (using Matplotlib), trend charts (using Altair), and a clear option for image uploads, ensuring easy interaction for users of all technical levels.
- **Data Storage and Analysis:** Stores user walking data in CSV files using Pandas, enabling long-term trend analysis and proactive health management through historical insights into walking patterns.

The proposed system enhances user safety, fall prevention, and proactive healthcare by leveraging step count-based analysis, a rule-based stability scoring mechanism, and real-time data analytics. By providing immediate fall risk alerts through the interface and personalized health guidance via the AI chatbot, this solution aims to reduce fall-related emergencies and improve healthcare accessibility, making it a reliable and efficient health monitoring tool. Additionally, the system is designed with the potential for future enhancements, such as integrating machine learning models like Support Vector Machines (SVM) to further improve the accuracy of fall risk predictions.

2.3. Feasibility Study:

Technical Feasibility

The "Health Monitoring System with Real Time Alerts" is built using a combination of modern technologies to ensure efficient development and operation. Streamlit is used for creating an intuitive user interface, featuring components like image uploaders, visualizations (e.g., stability meter with Matplotlib, trend charts with Altair), and a clear option for image uploads.

Python serves as the backend language, handling data processing, image analysis with Tesseract OCR (via pytesseract), and CSV file management with Pandas. The Google Gemini AI API is integrated to power the AI chatbot, providing personalized health recommendations.

While the current implementation uses a rule-based stability scoring mechanism for fall risk prediction, the system is designed to support future integration of machine learning models using Scikit-learn (e.g., Support Vector Machines for enhanced fall risk prediction).

The project requires a minimum system configuration of 4GB RAM and an Intel i3 processor to support real-time processing, image analysis, and data visualization. Software requirements include Python, Streamlit, pytesseract, Pillow (for image processing), Pandas, Matplotlib, Altair, and the Google Gemini AI API.

These technologies are widely supported and well-documented, ensuring the system's technical feasibility for development and deployment.

Economic Feasibility

The development cost of the "Health Monitoring System with Real Time Alerts" is minimal, as it primarily relies on open-source technologies such as Python, Streamlit, Pandas, Matplotlib, Altair, and pytesseract. Tesseract OCR is also open-source, requiring only a one-time setup with no licensing fees.

The Google Gemini AI API may incur usage-based costs, depending on the volume of chatbot interactions, but these can be managed by optimizing API calls. Hosting can be achieved cost-effectively using platforms like Streamlit Community Cloud (free for public apps) or cloud services such as AWS, Google Cloud Platform (GCP), or Azure, depending on budget constraints.

The system is designed to be a cost-effective solution for walking stability monitoring and fall risk prediction, eliminating the need for expensive wearable devices or sensors by leveraging image-based data collection (e.g., from smartphone apps like Samsung Health).

It benefits users, caregivers, and healthcare providers by providing real-time fall risk alerts and health guidance, potentially reducing healthcare costs associated with fall-related injuries and hospitalizations.

Operational Feasibility

The "Health Monitoring System with Real Time Alerts" is designed for users, including patients, caregivers, and healthcare professionals, with a focus on ease of use and accessibility.

The system features a user-friendly Streamlit-based web interface that allows users to upload images of step count data, manually input walking metrics, and view real-time fall risk predictions through intuitive visualizations like a stability meter and trend charts.

The integration of an AI-powered chatbot, using the Google Gemini AI API, provides instant health recommendations and answers to user queries, making the system accessible to individuals with limited technical knowledge.

Users can easily interact with the system to monitor walking stability, receive fall risk alerts, and access health guidance without requiring extensive technical expertise.

The system ensures seamless operation by automating data extraction (via OCR), stability scoring, and feedback delivery, making it a practical tool for proactive health management and fall prevention.

Legal Feasibility

The "Health Monitoring System with Real Time Alerts" must comply with data privacy regulations such as HIPAA (in the U.S.) or GDPR (in the EU) to ensure the security and confidentiality of user data.

Although the system currently stores data in CSV files, which are not inherently secure, future implementations should adopt secure storage solutions (e.g., encrypted databases) to protect user information, including step count data and chatbot interactions.

Legal considerations include obtaining user consent for data collection, particularly for walking data extracted from images and manually entered metrics. The system architecture must prevent unauthorized access and data breaches, ensuring that user data is handled responsibly.

The Google Gemini AI API, used for the chatbot, employs secure communication protocols, but the system must ensure that API interactions comply with privacy standards. By addressing these legal requirements, the system can maintain user trust and operate within regulatory frameworks.

2.4 System Requirements

The Health Monitoring and Alert System requires both functional and non- functional specifications to ensure efficient real-time monitoring, AI-driven decision-making, and emergency alert mechanisms.

2.4.1 Functional Requirements

Functional requirements define the core functionalities that the system must implement to full fill user needs. The key functional requirements include:

1. Login/Register

- Users must be able to securely register and log in to the Health Monitoring System.
- The system must manage user authentication and ensure data privacy.
- Login/Register is a prerequisite to access the dashboard and other features.

2. Dashboard

- The dashboard will serve as the central platform for users and caretakers to observe health data and access system modules.
- It will display health parameter trends and allow navigation to other components like data input, fall prediction, and guidelines.
- Includes real-time monitoring of user health inputs and alerts.

3. Health Data Input

- Users can manually enter or upload health parameters such as:
 - a) Blood Pressure (BP)
 - b) Heart Rate Variability (HRV)
 - c) Blood Sugar Levels
 - d) Oxygen Saturation (SpO2)

4. Fall Prediction

- The system will analyze the health data input using machine learning models (SVM, Logistic Regression, KNN, Naïve Bayes, Random Forest).
- It will predict potential falls or health deterioration and guide users accordingly.
- Provides real-time prediction feedback on the dashboard.

5. Alert Care Taker

- Upon detecting critical conditions or a fall, the system will automatically alert caretakers.
- Alerts will be sent via SMS/Call using Twilio API.
- Ensures timely response in emergencies.

6. Health Guidelines

- Based on health input and predictions, the system will provide personalized health guidelines.
- Guidelines will include preventive tips, lifestyle advice, and next steps.
- Displayed on dashboard and available through chatbot interaction.

7. Chatbot

- A built-in AI-powered chatbot will help users with:
 - a) Health queries
 - b) Accessing guidelines
 - c) System navigation support
- Integrated with Google Gemini API for intelligent and context-aware responses.

Functional Requirements

S.No	Requirement ID	Requirement Name	Requirement Description	Essential/Desirable
1	FR-01	Login/Register	Users must be able to register and log in securely.	Essential
2	FR-02	Dashboard	Provides an overview of user health data and system functionalities.	Essential
3	FR-03	Health Data Input	Users should be able to manually enter health parameters like BP, HRV, Sugar Levels, and SpO2.	Desirable
4	FR-04	Fall Prediction	Analyzes health data to detect falls and potential risks.	Essential
5	FR-05	Alert Care Taker	Automatically sends SMS and call alerts via Twilio API during critical situations.	Essential
6	FR-06	Health Guidelines	Provides users with health recommendations and preventive care advice.	Essential
7	FR-07	Chatbot	Provides real-time health advice and recommendations using Google Gemini AI.	Essential

Table 1.1 Functional Requirements
(SRS)

2.4.2 Non-Functional Requirements

Non-functional requirements ensure the system's performance, security, and usability. These include:

1. Performance Requirements

- The system should process health data in real-time without significant delays.
- Machine learning models must analyze and predict health conditions within seconds.

2. Scalability

- The system should be scalable to handle multiple users simultaneously.
- It must support increasing amounts of health data storage and processing.

3. Security & Privacy

- User health data must be encrypted and securely stored.
- Secure authentication mechanisms must be implemented to prevent unauthorized access.

4. Reliability & Availability

- The system should have 99.9% uptime and be available 24/7.
- SMS and call alerts must be delivered reliably without failure.

5. Usability & Accessibility

- The Streamlit UI must be simple and intuitive for users with minimal technical knowledge.
- The system should be accessible for elderly and differently-abled users.

6. Maintainability & Upgradability

- The system should be modular so that updates and new features can be easily integrated.
- Code should follow best practices for maintainability

Non-Functional Requirements

S.No	Requirement ID	Requirement Name	Requirement Description	Essential/Desirable
1	NFR-01	Performance	The system should process health data and generate alerts within seconds.	Essential
2	NFR-02	Security	Encrypt user data and implement secure authentication mechanisms.	Essential
3	NFR-03	Scalability	The system should support multiple users and increasing health data.	Desirable
4	NFR-04	Reliability	Ensure 99.9% uptime and reliable alert delivery.	Essential
5	NFR-05	Accessibility	UI should be simple and usable for elderly and differently-abled users.	Desirable

Table 1.2 Non-Functional Requirements
(SRS)

Software and Hardware Requirements

The Health Monitoring System with Real Time Alerts is a sophisticated application that integrates real-time health tracking, fall risk prediction, AI-driven assistance, and emergency alert mechanisms. To ensure its seamless development, deployment, and operation, a well-defined set of software and hardware requirements has been established. These requirements are meticulously crafted to support the system's functional needs—such as continuous health parameter monitoring, fall detection, and user interaction—while adhering to the non-functional specifications of performance, scalability, security, and usability outlined in Section 2.4.2. The following subsections detail the software and hardware components essential for the system's success, providing a comprehensive foundation for its implementation and future enhancements.

Software Requirements

The system relies on a robust software stack to handle data processing, user interface rendering, machine learning predictions, and communication functionalities. The following software components are essential:

- **Operating System:** Windows 10/11, Linux (Ubuntu 20.04 or later), or macOS (12.0 Monterey or later) to support development and deployment across diverse platforms.
- **Programming Language:** Python 3.9 or higher, chosen for its extensive libraries and ease of integration with machine learning and web frameworks.
- **Web Framework:** Streamlit 1.15 or later, utilized for creating an interactive, user-friendly web interface with real-time data visualization capabilities.
- **Machine Learning Library:** Scikit-learn 1.2 or higher, employed for implementing Support Vector Machine (SVM), Logistic Regression, KNN, Naïve Bayes, and Random Forest models for health risk prediction.
- **Data Management:** Pandas 1.5 or higher, used for efficient CSV-based storage, data manipulation, and long-term trend analysis.
- **Image Processing and OCR:** Tesseract OCR 5.0 or later with pytesseract wrapper, required for extracting step counts and dates from uploaded images (e.g., Samsung Health screenshots).
- **Visualization Libraries:** Matplotlib 3.6 or higher for stability meter visualization and Altair 5.0 or higher for interactive trend charts, enhancing user interaction with health data.
- **Communication APIs:** Twilio API (Python SDK 8.0 or later) for SMS and call alerts, and Google Gemini AI API (latest version as of April 2025) for powering the AI chatbot with personalized health recommendations.
- **Text-to-Speech:** pyttsx3 2.90 or higher, enabling offline speech alerts for accessibility.
- **Development Tools:** Visual Studio Code 1.75 or later for coding, Jupyter Notebook 6.5 or higher for prototyping, and Google Colab (cloud-based) for machine learning experimentation with GPU support.

Hardware Requirements

The hardware specifications ensure that the system can process real-time data, perform machine learning predictions, and handle user interactions efficiently. The minimum and recommended configurations are as follows:

- **Processor:** The minimum requirement is an Intel Core i3 (8th Generation) or equivalent AMD processor, capable of handling basic Python execution, Streamlit rendering, and CSV file operations. For optimal performance, especially during real-time data processing and potential machine learning inference, an Intel Core i5 (10th Generation) or higher is recommended. This ensures smooth execution of tasks like OCR processing, stability score calculations, and simultaneous API calls without lag.
- **RAM:** Minimum - 4 GB to support basic operations; Recommended - 8 GB or higher to handle real-time monitoring, image processing, and multi-user access without performance degradation.
- **Storage:** A minimum of 256 GB HDD or SSD is required to store the software stack (Python, libraries, Streamlit), CSV files containing user data, and temporary image uploads. A 512 GB SSD is recommended for faster read/write speeds, which are critical when frequently accessing and updating health records for trend analysis. SSDs also improve the system's boot time and overall performance, making them ideal for deployment on personal or server hardware.
- **Graphics:** Integrated graphics (e.g., Intel UHD Graphics) sufficient for visualization; no dedicated GPU required unless machine learning models are trained locally (optional NVIDIA GTX 1650 or higher).
- **Network:** A stable internet connection with a minimum speed of 5 Mbps is essential for interacting with the Twilio API (SMS/call alerts) and Google Gemini AI API (chatbot responses). This ensures timely delivery of emergency notifications and real-time health advice. The system supports offline mode for core functionalities like stability scoring and speech alerts, but full functionality requires connectivity. A recommended speed of 10 Mbps or higher supports smoother operation, especially for cloud-hosted deployments.
- **Display:** A minimum resolution of 1366x768 is required to view the Streamlit interface comfortably, including widgets, charts, and text inputs. A 1920x1080 (Full HD) resolution is recommended for optimal clarity and detail, particularly when displaying multiple visualizations (e.g., stability meter alongside trend charts) or interacting with the AI chatbot. This ensures a visually appealing and functional experience for users.
- **Input Devices:** A standard keyboard and mouse are necessary for manual data entry (e.g., health parameters, user queries) and navigation. An optional webcam or smartphone camera (minimum 5 MP) is required for capturing step count images, such as Samsung Health screenshots, which are processed via Tesseract OCR. This flexibility allows users to leverage existing devices, enhancing accessibility without mandating specialized hardware.

3. SYSTEM DESIGN

3.1. Object Oriented Analysis and Design:

Introduction to OOAD

Object-Oriented Analysis and Design (OOAD) is a software engineering approach used to model a system as a group of interacting objects, each with its own set of properties and behaviors. It focuses on defining the structure and behavior of a system through the identification of objects, their relationships, and the interactions between them. OOAD promotes modular design, reusability, and maintainability by encapsulating related functionality within objects and defining clear interfaces for communication between objects.

Definition of OOAD

Object-Oriented Analysis and Design (OOAD) is a methodology for designing software systems based on the principles of object-oriented programming (OOP). It involves analyzing the requirements of a system, identifying objects in the problem domain, defining their attributes and behaviors, and modeling their interactions to develop a robust and scalable software solution. OOAD emphasizes abstraction, encapsulation, inheritance, and polymorphism as fundamental concepts for designing modular and extensible software systems.

Purpose of Object-Oriented Analysis and Design

The purpose of Object-Oriented Analysis and Design (OOAD) is to:

- **Facilitate understanding:** OOAD helps stakeholders understand the system requirements by modeling them using familiar object-oriented concepts.
- **Encourage modularity:** OOAD promotes modular design by encapsulating related functionality within objects and defining clear interfaces for communication between objects.
- **Enhance reusability:** OOAD facilitates the creation of reusable software components, reducing development time and effort for future projects.
- **Improve maintainability:** OOAD leads to software designs that are easier to maintain and update, as changes can be localized to specific objects or classes.
- **Support scalability:** OOAD enables the development of scalable software systems that can accommodate changes in requirements and scale to meet growing user demands.

3.1.1 Scenarios

- The following table outlines real-world scenarios where the Health Monitoring and Alert System can be effectively utilized.

Scenario 1: Login/Register

Use Case Scenario	Login/Register
Participating Actors	User, System
Pre-conditions	User accesses the system's login or registration page
Flow of Events	1. User enters credentials or registration details. 2. System verifies input. 3. User is authenticated or newly registered.
Expected Outcome	User is granted access to the system.
Alternative Outcome	System rejects incorrect credentials or registration details.
Post-conditions	User is logged in or registered.
Alternative Post-conditions	User retries or contacts support.

Scenario 2: Dashboard

Use Case Scenario	Dashboard
Participating Actors	Patient, System
Pre-conditions	User is authenticated and logged in
Flow of Events	1. System displays health summary, graphs, alerts, and recent data. 2. User navigates through health statistics and status. 3. System updates values in real-time.
Expected Outcome	User receives a visual summary of health status.
Alternative Outcome	System fails to load dashboard due to technical issues.
Post-conditions	User sees updated health records and system insights.
Alternative Post-conditions	System logs error and prompts retry.

Scenario 3: Health Data Input

Use Case Scenario	Health Data Input
Participating Actors	Patient, System
Pre-conditions	Patient is logged in
Flow of Events	1. Patient inputs BP, HRV, SpO2, or Sugar level data manually. 2. System validates and stores the data. 3. Updates are reflected in graphs and analysis.
Expected Outcome	Health data is stored and used for trend analysis.
Alternative Outcome	Invalid input is rejected by system.
Post-conditions	New data is reflected in patient record.
Alternative Post-conditions	System alerts user to re-enter valid data.

Scenario 4: Fall Prediction

Use Case Scenario	Fall Prediction
Participating Actors	Elderly Patient, System
Pre-conditions	Wearable device is active and patient is registered
Flow of Events	1. Wearable detects abnormal motion patterns. 2. System analyzes fall probability. 3. If risk is high, emergency protocol is triggered. 4. Alert sent to caregiver.
Expected Outcome	Fall is detected or predicted early, enabling timely intervention.
Alternative Outcome	False positive or device fails to detect the fall.
Post-conditions	Alert and fall details stored; caregiver responds.
Alternative Post-conditions	No alert is triggered, but system logs data for analysis.

Scenario 5: Alert Care Taker

Use Case Scenario	Alert Care Taker
Participating Actors	Patient, Caregiver, System
Pre-conditions	Patient's vitals are monitored by system
Flow of Events	1. System detects abnormal vitals. 2. Alert is generated. 3. SMS/call is sent to caregiver and patient. 4. Caregiver takes action remotely.
Expected Outcome	Alert successfully reaches caregiver and patient.

Use Case Scenario	Alert Care Taker
Alternative Outcome	SMS/call fails or delay in notification.
Post-conditions	Alerts logged; patient and caregiver act accordingly.
Alternative Post-conditions	Retry or alternative contact methods used.

Scenario 6: Health Guidelines

Use Case Scenario	Health Guidelines
Participating Actors	Doctor, Patient, System
Pre-conditions	Doctor or AI has access to patient history
Flow of Events	1. Doctor analyzes health data via dashboard. 2. System generates trends or AI assists. 3. Personalized guidelines or recommendations are issued. 4. Patient views advice in dashboard.
Expected Outcome	Patient receives tailored health recommendations.
Alternative Outcome	Data unavailable or incomplete for recommendation.
Post-conditions	Guidelines are stored for future access.
Alternative Post-conditions	Patient is advised to update records before re-analysis.

Scenario 7: Chatbot

Use Case Scenario	Chatbot
Participating Actors	User, AI Chatbot, System
Pre-conditions	User is logged in and has recent health data
Flow of Events	1. User interacts with chatbot through UI. 2. Chatbot accesses user data. 3. System uses ML to provide personalized advice. 4. User continues dialog for further clarification.
Expected Outcome	User receives relevant health advice and guidance.
Alternative Outcome	Chatbot fails to interpret or process user data.
Post-conditions	Conversation is logged; user applies advice.
Alternative Post-conditions	User is redirected to contact a real medical professional.

Table 2.0 Scenario

UML Diagrams

UML diagrams are classified into three main categories:

- **Structural Diagrams:** Depict the static structure of a system. Examples include Class Diagrams, Object Diagrams, Component Diagrams, and Package Diagrams.
- **Behavioural Diagrams:** Describe the dynamic behaviour of a system. Examples include Use Case Diagrams, Sequence Diagrams, Activity Diagrams, and State Machine Diagrams.
- **Interaction Diagrams:** Focus on the interactions between objects or components within the system. Examples include Sequence Diagrams and Communication Diagrams.

3.1.2 UseCase Diagram:

Health Monitoring System allows users to monitor health parameters, predict fall risks, and alert caregivers in case of emergencies. It provides real-time health tracking, fall detection, emergency alerts, chatbot-based guidance, and a user-friendly dashboard.

Actors:

1. **User:** The primary user who logs in, enters health data, and receives health insights.
2. **Care Taker:** A secondary actor who observes user health data and receives alerts in case of emergencies.

Use Cases:

1. Login/Register:

- The user registers and logs into the system to access health monitoring features.

2. Dashboard (<> Login/Register):

- After login, users can access the dashboard to navigate different system functionalities.

3. Health Data Input (<> Dashboard):

- Users enter their health parameters (BP, HRV, Sugar Levels, SpO2).
- Data is used for fall prediction and health analysis.

4. Fall Prediction:

- Predicts potential fall risks based on health data.
- If a fall is detected, an alert is triggered.

5. Alert Care Taker (<> Fall Prediction):

- If the system detects a critical health condition, it sends SMS and call alerts to the caregiver.

6. Health Guidelines:

- Provides users with personalized health recommendations.

7. Chatbot:

- Users can chat with an AI-powered chatbot to get health-related advice.
- The caregiver can also interact with the chatbot for user health insights.

Relationships:

1. Includes (<>):

- Dashboard is included in Login/Register since access requires authentication.
- Health Data Input is part of the Dashboard for managing health records.
- Fall Prediction uses Health Data Input to analyze health risks.
- Alert Care Taker is triggered by Fall Prediction if a risk is detected.

2. Associations:

- User and Care Taker both interact with the Health Monitoring System.
- The User observes and inputs health data, while the Care Taker observes and responds to alerts.
- The Chatbot and Health Guidelines assist both User and Care Taker.

3. Dotted Arrows:

- Represent predictions, guidelines, and interactions, such as:
 - Fall Prediction using health data.
 - Chatbot providing health guidance based on input.

This use case diagram represents the core functionalities of the Health Monitoring System, illustrating how users track their health, detect falls, and get emergency support through alerts and AI-driven recommendations.

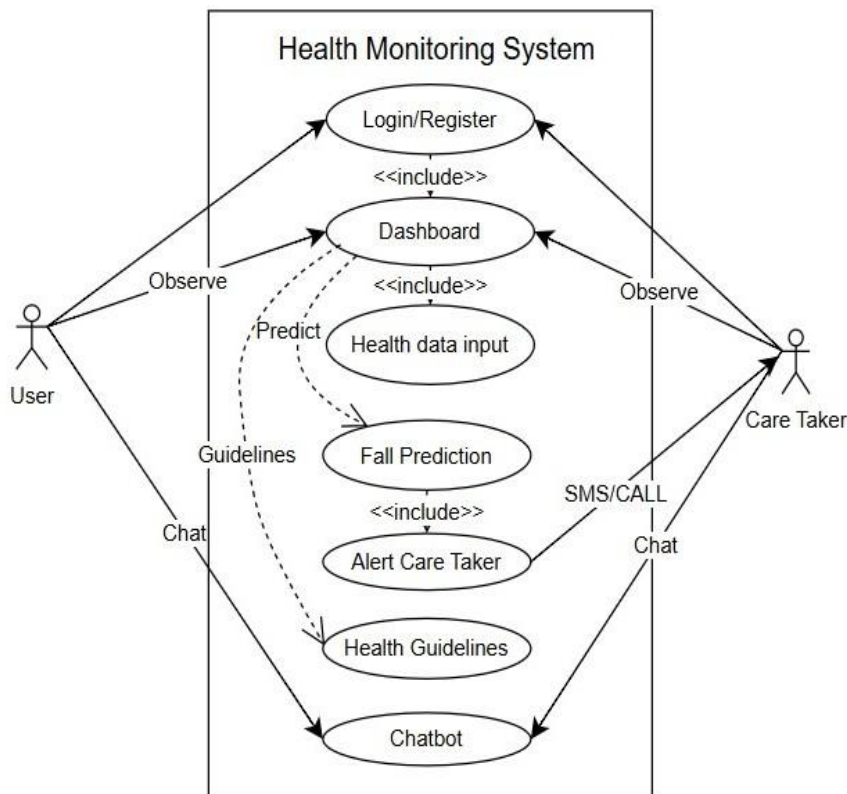


Fig 1.0 Use Case Diagram

3.1.3 Sequence Diagram

1. Login/Register

Description:

- This sequence diagram illustrates the process of user authentication. The user selects the login or register option, and the Health Monitoring System displays the appropriate form. The user submits their credentials (username, password, and additional details for registration), which are validated or stored in the data file. The system then returns the result (success or failure) and displays it to the user (e.g., "Logged in" or "Registration Failed").

Purpose: To ensure secure access to the system by allowing users to log in or register, verifying their credentials, and providing feedback on the authentication status.

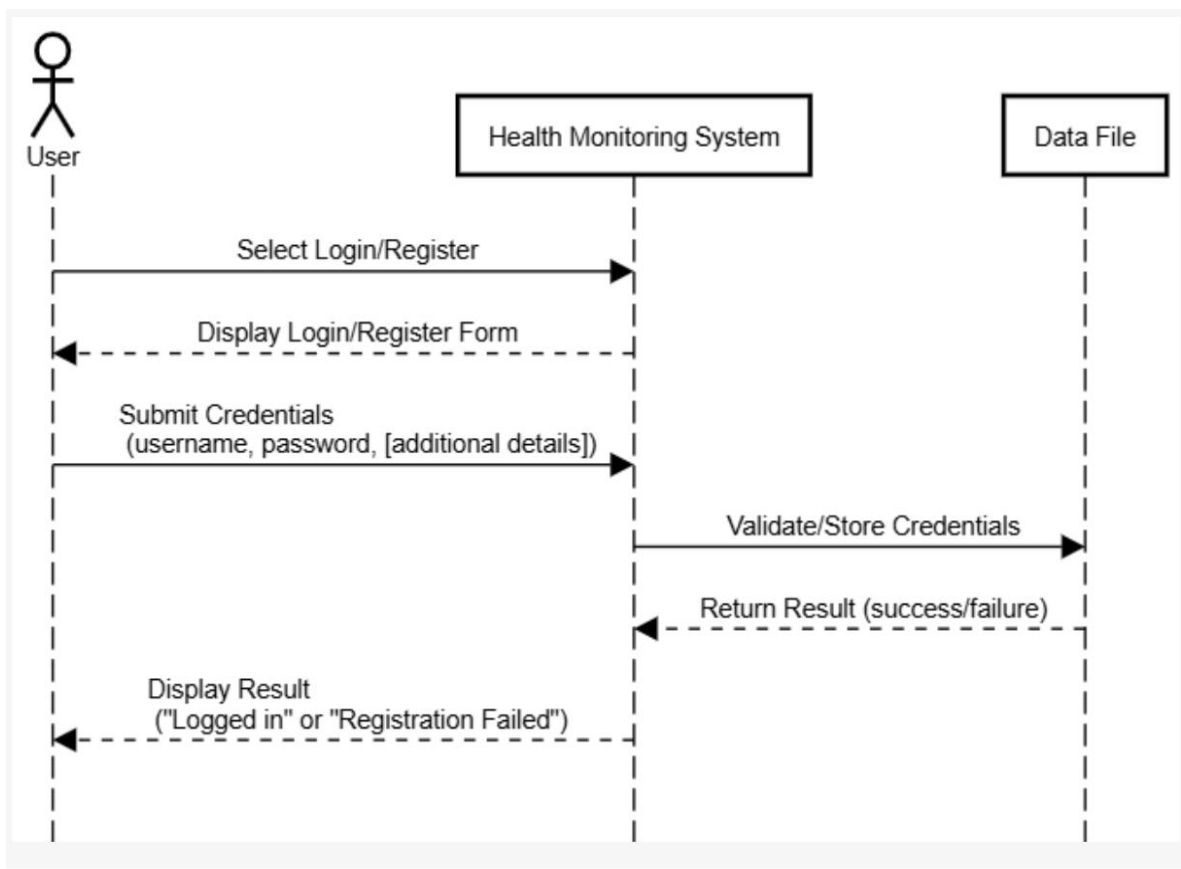


Fig 2.1 Sequence Diagram for
Login/Register

2. Dashboard

Description: This sequence diagram shows the user requesting the dashboard, prompting the Health Monitoring System to retrieve user health data from the data file, process it for display, and then present the dashboard with metrics and navigation options to the user.

Purpose: To provide users with a centralized view of their health metrics and navigation options, enabling them to monitor their health status and access other system features.

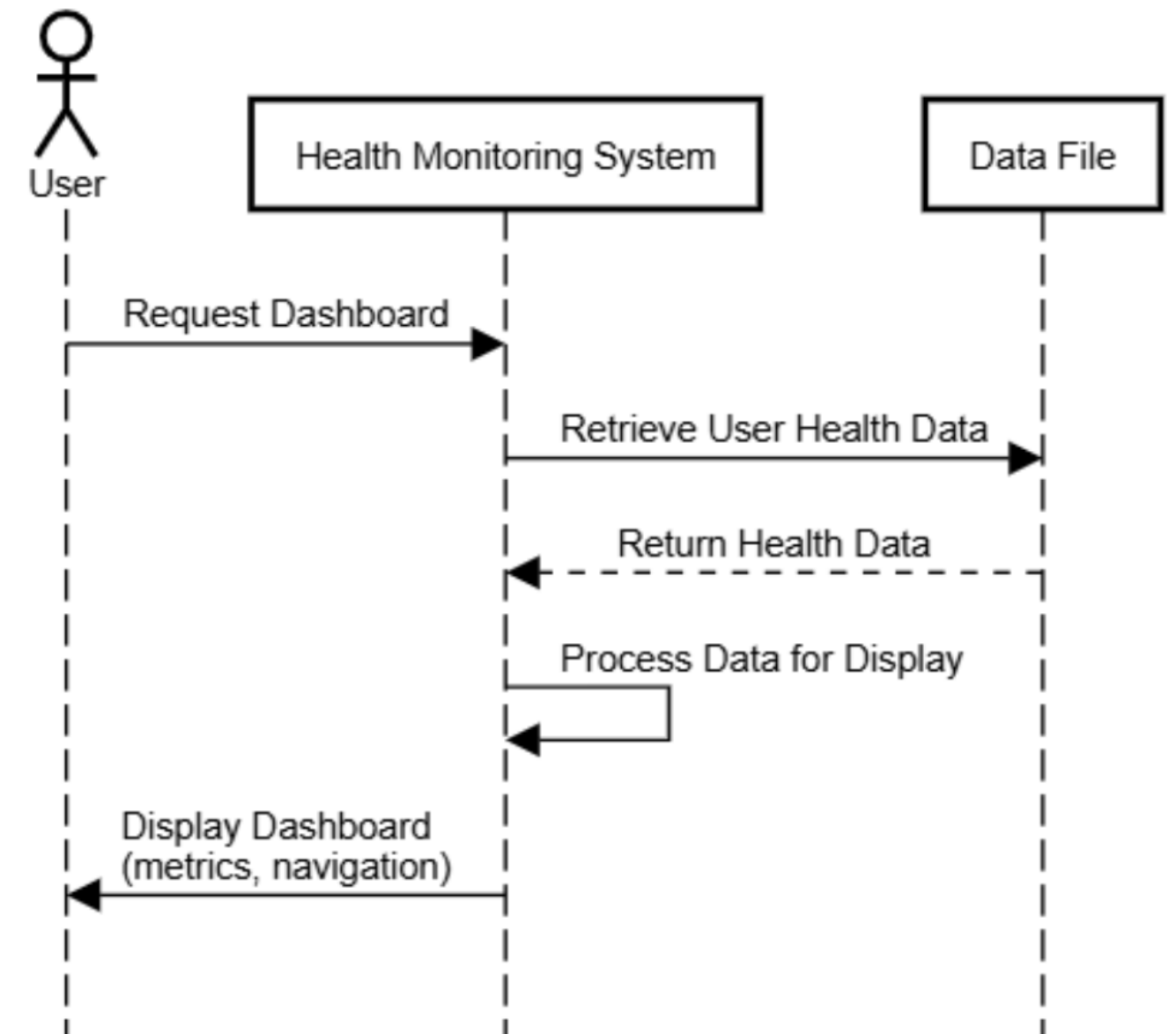


Fig 2.2 Sequence Diagram for
Dashboard

3. Fall Prediction

Description: This sequence diagram illustrates the process of the user navigating to the fall detection section and submitting health data for a specified number of days. The Health Monitoring System sends the data to a machine learning model for fall prediction, receives the prediction (Fall/No Fall), stores the result in the data file, and displays the fall prediction result to the user.

Purpose: To predict the likelihood of a fall based on user health data using a machine learning model, providing users with insights into potential fall risks for proactive prevention.

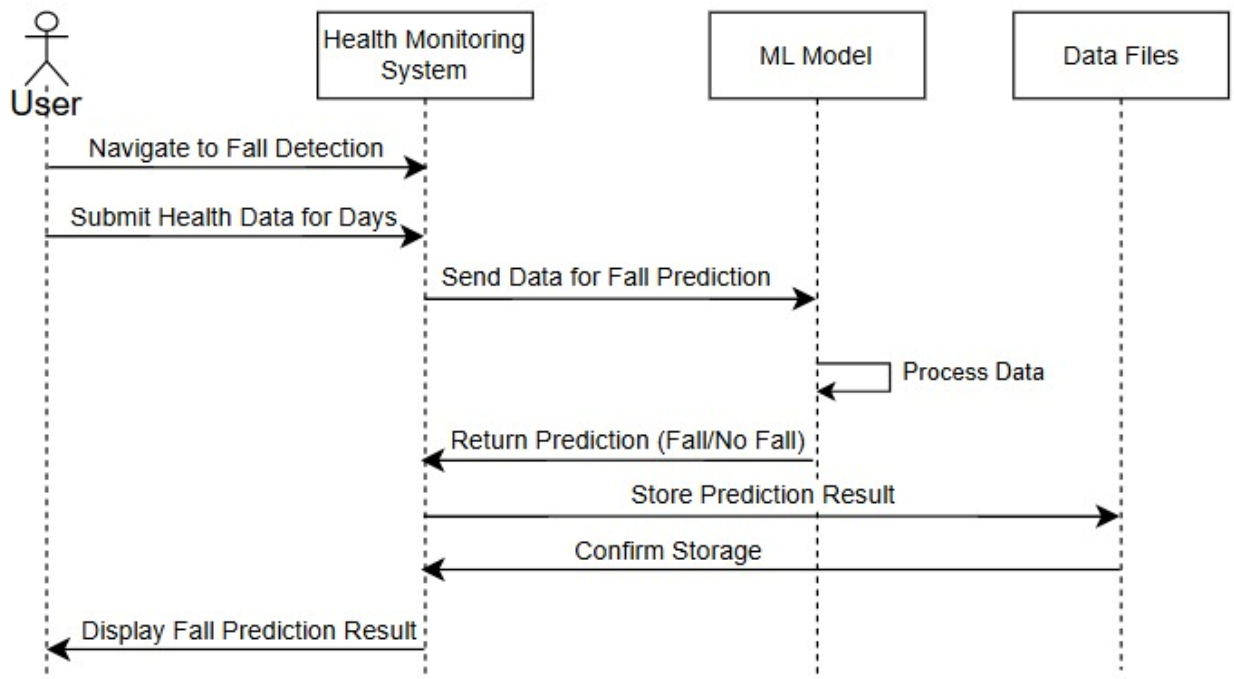


Fig 2.3 Sequence Diagram for
Fall Prediction

4. Alert Care Taker (SMS/Call)

Description: This sequence diagram shows the Health Monitoring System detecting a critical condition, sending SMS and call requests via the Twilio API to a caregiver, and displaying the alert status to the user after confirming the SMS and call have been initiated.

Purpose: To ensure timely notification of caregivers in emergency situations by automating SMS and call alerts when a critical health condition is detected.

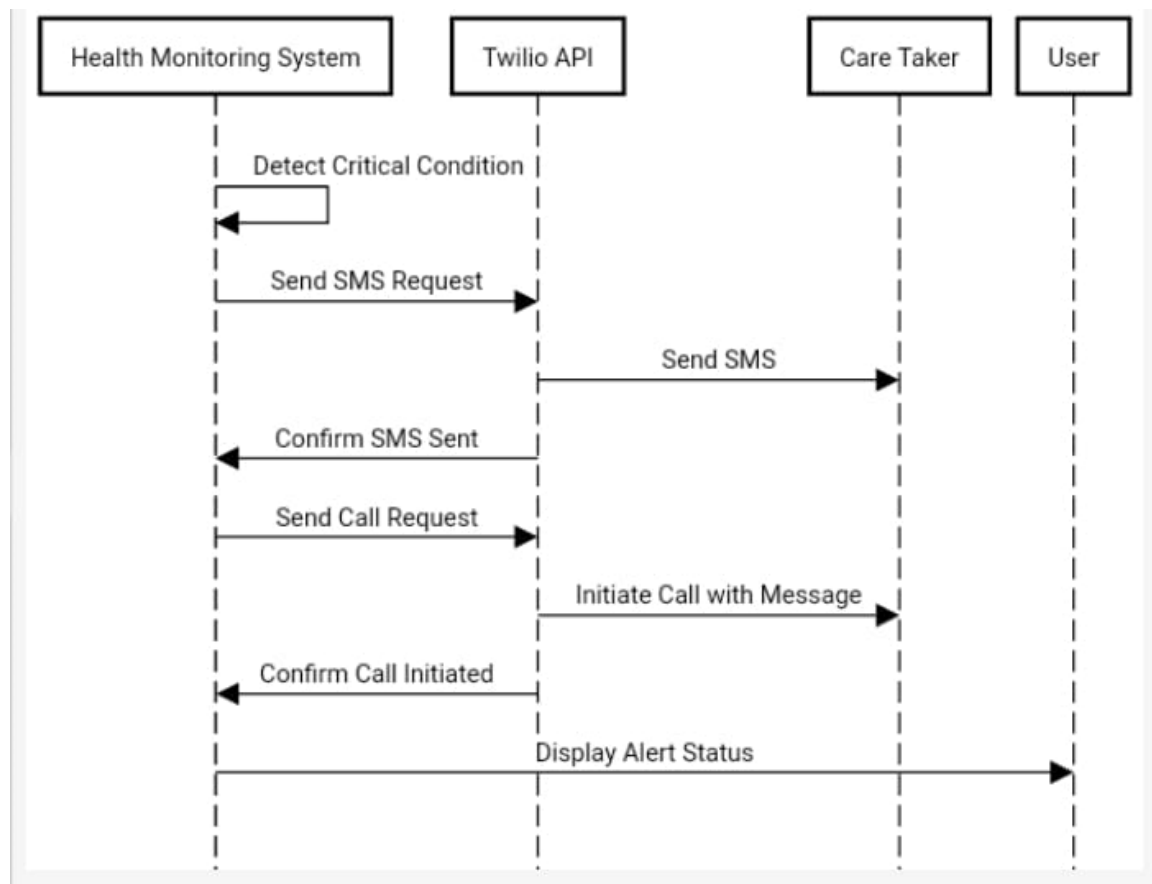


Fig 2.4 Sequence Diagram for
Alert Care Taker (SMS/Call)

5. Health Guidelines

Description: This sequence diagram shows the user navigating to a guidelines section, where the Health Monitoring System retrieves health data and predictions from the data file, generates personalized health guidelines, and displays them to the user.

Purpose: To provide users with personalized health guidelines based on their historical health data and predictions, aiding in proactive health management.

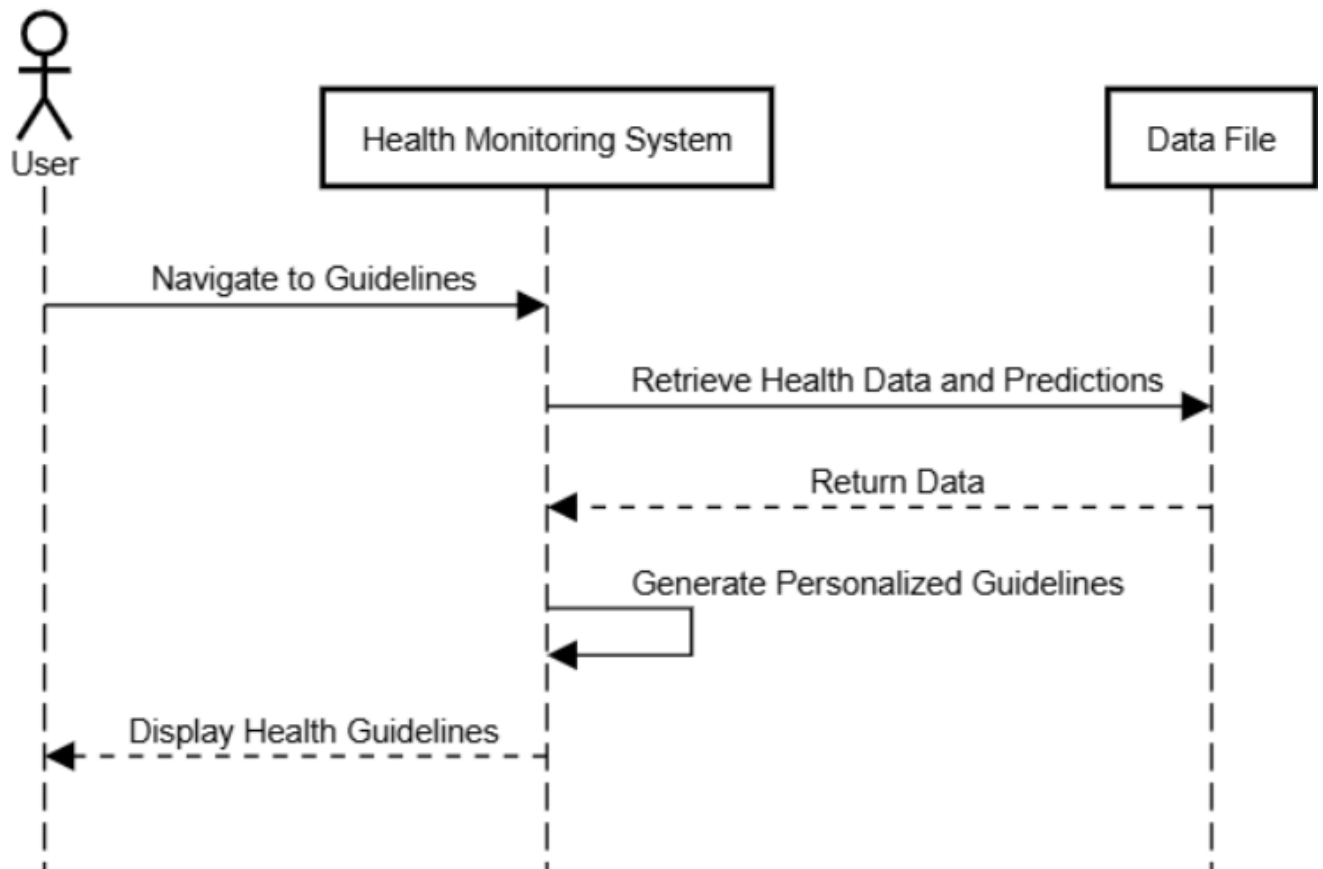


Fig 2.5 Sequence Diagram for
Health Guidelines

6. Chatbot

Description: This sequence diagram depicts the interaction between the user and the AI chatbot. The user navigates to the chatbot section, sends a query, and the Health Monitoring System forwards the query to the Gemini API. The API processes the query, returns a response, and the system displays the response to the user while logging the chat in the session state.

Purpose: To provide users with real-time health recommendations and guidance through an AI-powered chatbot, enhancing accessibility to health advice, especially for users with limited technical knowledge.

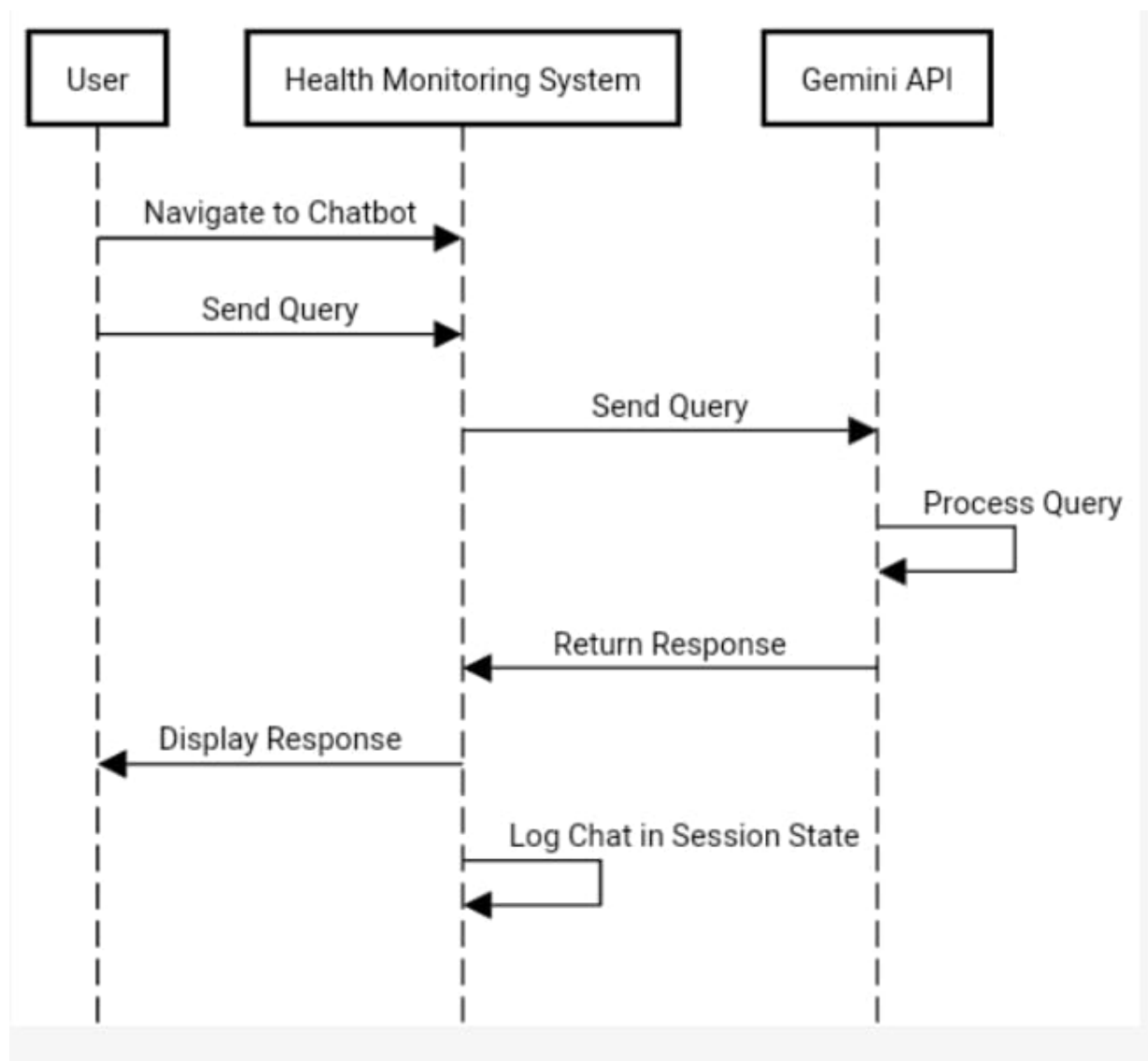


Fig 2.6 Sequence Diagram
for Chatbot

3.1.4 Class Diagram

This class diagram represents the structure of the Health Monitoring System, showing the relationships between classes, their attributes, and methods. It illustrates how users interact with the system, how health data is monitored, and how alerts are generated and sent to caretakers.

Classes and Their Descriptions:

1. System Class

- **Attributes:**

- system: string – Represents the system's name or type.
- mlaccuracy: float – Stores the accuracy of the machine learning model

- **Methods:**

- predictFall(): boolean – Predicts whether a fall might occur.
- healthconditions(): boolean – Checks the health condition of the user.
- healthguideline(): string – Provides health-related guidelines
- chatbot(): string – Enables chatbot interaction.

Relationships:

- Used by User and Caretaker (1 to 0..* relation)
- Monitors HealthMonitoring (1 to 1 relation).

2. User Class

- **Attributes:**

- userID: int – Unique identifier for the user.
- name: String – Name of the user.
- age: int – Age of the user.
- email: String – Email of the user.
- password: String – User's password.

- **Methods:**

- register(): boolean – Allows a new user to register.
- login(): boolean – Allows an existing user to log in.

Relationships:

- Uses the System (1 to 0..* relation).
- Can access the Dashboard for health reports.

3. Dotted Arrows:

- **Methods:**

- displaySummary(): boolean – Displays an overview of health data
- viewReports() – Allows users to view their health reports
- viewHealthData(): boolean – Enables users to see their stored health records.

Relationships:

- Connected to User for displaying health data.

4.HealthMonitoring Class

- **Attributes:**

- healthData: int – Stores health monitoring data.
- bloodPressure: String – Records blood pressure readings.

- **Methods:**

- trackHealthRate(): int – Tracks health conditions in real-time.

Relationships:

- The **System** monitors HealthMonitoring (1 to 1 relation).
- HealthMonitoring sends alerts to AlertSystem if critical conditions are detected

5.AlertSystem Class

- **Attributes:**

- alertID: String – Unique ID for each alert.
- alertMessage: String – Message describing the alert.
- CaretakerContact: String – Stores contact details of the caretaker.

- **Methods:**

- sendAlert(): void – Sends an emergency alert notification.
- emergencyCall(): boolean – Initiates an emergency call.

Relationships:

- Receives data from HealthMonitoring.
- Sends alerts to the Caretaker (1 to * relation).

6.Caretaker Class

- **Attributes:**

- caretakerID: int – Unique ID for the caretaker.
- name: String – Name of the caretaker.
- age: int – Age of the caretaker.
- email: String – Email of the caretaker.
- password: String – Caretaker's password.

- **Methods:**

- viewHealthData():boolean – Allows caretakers to view the health data of users
- receiveAlerts(): boolean – Enables caretakers to receive emergency alerts.

Relationships:

- The System monitors HealthMonitoring (1 to 1 relation).
- HealthMonitoring sends alerts to AlertSystem if critical conditions are detected

7.Chatbot Class

- **Attributes:**

- respondToQueries: boolean – Determines if the chatbot can respond to user queries
- provideGuidance: String – Provides health-related guidance.

- **Methods:**

- sendAlert(): void – Can trigger an alert if needed.

Relationships:

- Interacts with the User to provide health advice.

Key Relationships:

1. **User and System:** Users interact with the system to track health conditions.
2. **HealthMonitoring and AlertSystem:** If critical health conditions are detected, an alert is sent.
3. **AlertSystem and Caretaker:** Caretakers receive notifications and take action.
4. **User and Chatbot:** The chatbot assists users with health-related queries.

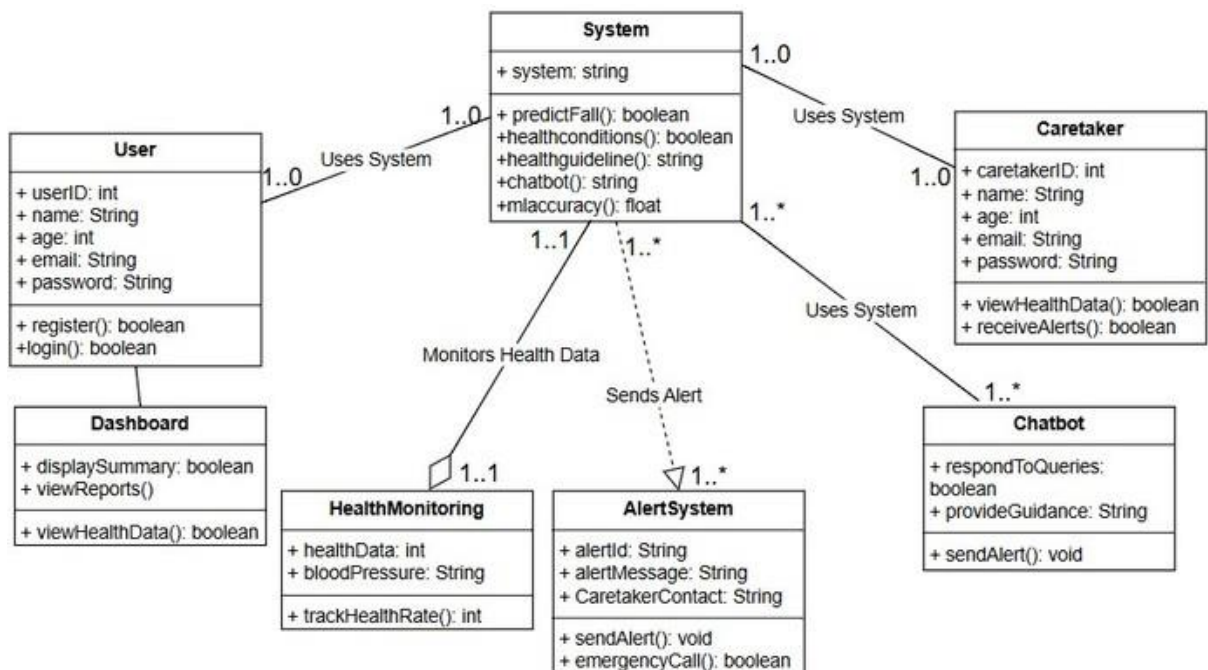


Fig 3.0 Class Diagram

3.1.5 Activity Diagram

This activity diagram represents the workflow of the Health Monitoring System, detailing user registration, health data input, analysis, health issue detection, emergency alerts, and chatbot interactions. It demonstrates the step-by-step process of how the system handles user data and takes appropriate actions based on health conditions.

Activity Flow Description:

1. Start:

- The process begins when a user registers or logs in to the system.

2. User Authentication:

- The system checks if the user is new or existing.
 - If new, the user is registered in the system.
 - If existing, the user proceeds to login.

3. Enter/Update User Details:

- Users update their personal details such as age, name, and medical history.

4. Input Health Data:

- Users enter their health parameters (Blood Pressure, HRV, Sugar Levels, SpO2).

5. Analyze Health Data:

- The system processes and analyzes the health data to identify risks.

6. Parallel Activities:

The system takes different actions based on the health analysis:

1. Health Issue Detection:

- If no health issue is detected → Continue Monitoring.
- If an issue is detected, the system predicts if a fall might occur
 - If a fall is predicted, the caretaker is notified via SMS/Call.
 - If no fall is predicted, the chatbot interacts with the user to provide guidance.

2. Health Checks & Report Generation:

- The system creates health reports based on the input data for user reference.

3. Chatbot Interaction:

- The AI chatbot provides health advice and guidance based on user conditions.

7. End of Process:

- The activity concludes when the health data is analyzed, actions are taken, and chatbot interactions are completed.

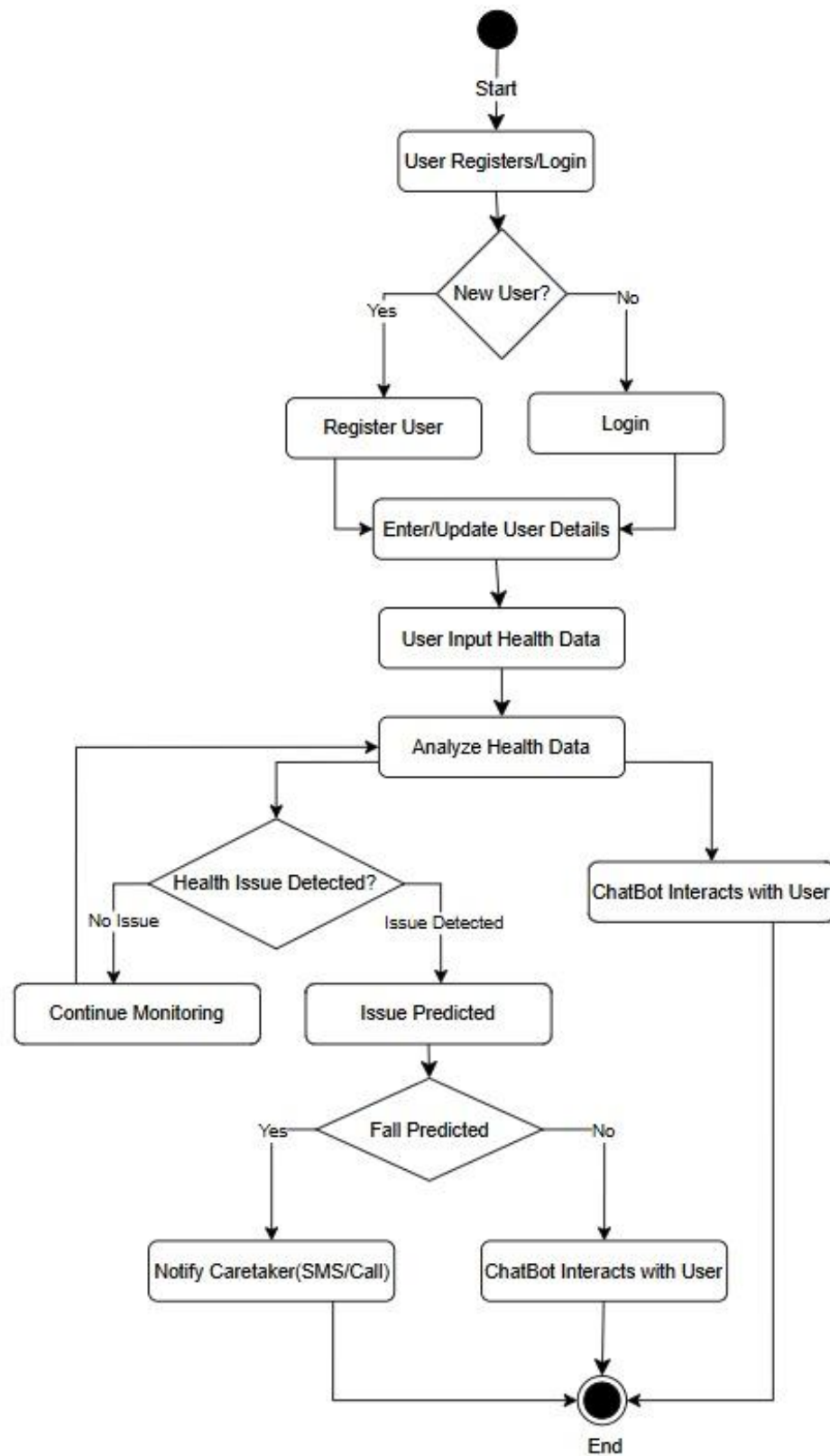


Fig 4.0 Activity Diagram

3.1.6 State Chart Diagram

Overview:

This state chart diagram represents the different states of the Health Monitoring System, showing how it transitions from one state to another based on user actions and system processes. It highlights user authentication, health data analysis, fall prediction, emergency alerts, and chatbot interactions.

State Transitions & Description:

1. Start State:

- The system begins in an initial state, waiting for user interaction.

2. User Access System:

- The user enters the system and is prompted to either register or log in.

3. Register/Login State:

- If the user is new, they register
- If the user already exists, they log in to proceed.

4. Health Data State:

- After successful authentication, the user enters their health parameters (BP, HRV, Sugar Levels, SpO2).

5. Analyze Health Data:

- The system processes the input data to check for health conditions or risks.

6. Parallel Processing States:

Based on the health analysis, the system can transition to multiple states:

- Fall Prediction:
 - If a fall risk is detected, the system triggers a notification alert via Call/SMS.
The caretaker is notified for emergency response.
- Continue Monitoring:
 - If no health issues are found, the system continues normal monitoring.
- Health Tips & Voice Response:
 - The system provides health tips based on the user's current condition.
- AI Chatbot Response:
 - If the user interacts with the chatbot, it processes queries and provides health-related guidance.

7. End State:

- The process reaches a final state after completing health monitoring, notifications, or chatbot interactions.

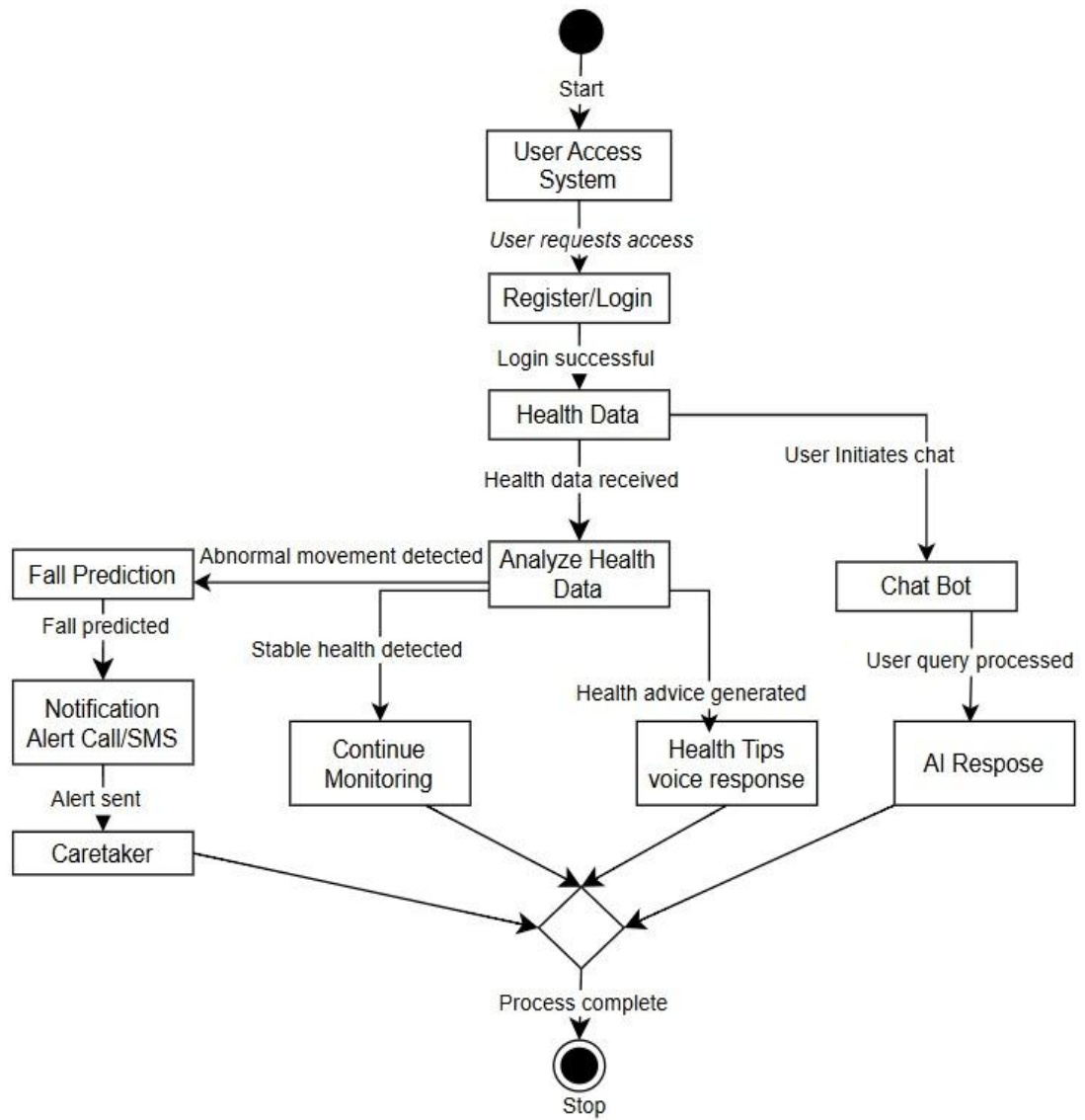


Fig 5.0 State Chart Diagram

3.1.7 Component Diagram

Overview:

This component diagram represents the architecture of the Health Monitoring System, illustrating how different modules and components interact with each other. It shows how user authentication, health monitoring, fall prediction, chatbot services, emergency alerts, and machine learning models work together using CSV-based data storage.

Key Components and Their Descriptions:

1. Web App (Main Application)

Prediction Module – Uses machine learning to predict fall risks.

Data Processing – Handles health data for analysis and prediction.

Alert System – Sends alerts to caregivers in case of emergencies.

Chatbot Module – Allows interaction with an AI-driven health assistant.

User Authentication – Manages user login and registration

Interacts With:

- **ML Model** for predictions
- **Data (CSV) storage** for user health data
- **Twilio API** for alerts
- **Chatbot** for AI-based responses

2. ML Model (Machine Learning Engine)

- **Fall Prediction Algorithm** – Detects fall risks based on user health data.
- **Feature Extraction** – Extracts relevant features for fall prediction.
- **Data Preprocessing** – Cleans and normalizes data before analysis.

Interacts With:

- **Web App's Prediction Module** to provide fall predictions.
- **Data (CSV)** for retrieving user health records.

3. Data (CSV Storage)

- **User Data** – Stores registered user details.
- **Health Records** – Stores past and real-time health data

Interacts With:

- **ML Model** for health predictions.
- **Web App (Dashboard)** for displaying health insights to users.

4. Twilio API (Emergency Alert System)

- **SMS Alerts** – Sends emergency messages to caretakers.
- **Call Alerts** – Initiates automated calls during emergencies.

Interacts With:

- Web App (Alert System) to notify caretakers.

5. Chatbot (AI-Based Health Assistant)

- **AI Response Engine** – Processes user queries and provides health advice.
- **Health Tips** – Offers personalized health recommendations.

Interacts With:

- Web App (Chatbot Module) to communicate with users.

6. Dashboard (User Interface)

- Displays user health records retrieved from Data (CSV).

Interacts With:

- Web App for user-friendly health monitoring.

Overall System Flow:

1. User logs in/registers through the Web App.
2. User inputs health data, which is stored in CSV files.
3. The ML Model processes the data and predicts fall risks.
4. If a fall is predicted, the Alert System sends SMS/Call notifications via Twilio API.
5. The Chatbot assists users by providing AI-generated health tips.
6. Users can view their past health data through the Dashboard.

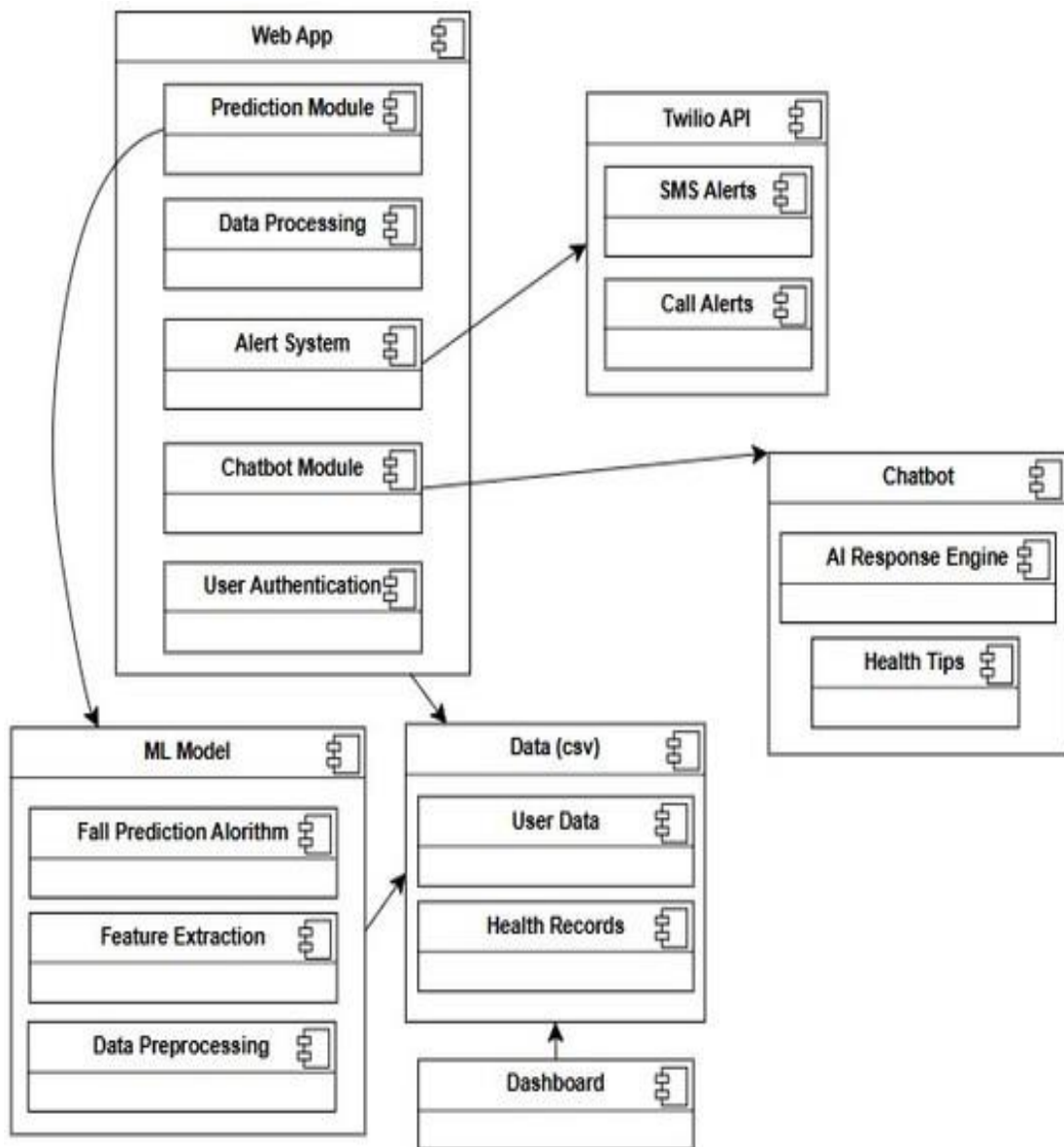


Fig: 6.0 Component Diagram

3.2 Database Design

Since this project uses CSV-based storage instead of a relational database, data is stored in structured CSV files, ensuring easy access, modification, and retrieval without requiring complex database management.

Data Storage Structure

- The system maintains multiple CSV files to store different types of information:
 - 1. User Data (users.csv)** – Stores patient registration details.
 - **Columns:** Username, Password, Age, Blood Group, Food Allergies
 - 2. Health Records (health_data.csv)** – Stores real-time health monitoring data.
 - **Columns:** Username, BP, HRV, Sugar Levels, SpO2, Timestamp
 - 3. Fall Detection Logs (fall_detection.csv)** – Stores records of detected falls.
 - **Columns:** Username, BP, HRV, Sugar Levels, SpO2, Fall Status, Timestamp
 - 4. Chatbot Interaction Logs (chatbot_logs.csv)** – Stores user queries and AI responses.
 - **Columns:** Username, Query, AI Response, Timestamp
 - 5. Alert Logs (alerts.csv)** – Stores records of SMS and call alerts triggered via Twilio.
 - **Columns:** Username, Alert Type (SMS/Call), Message, Timestamp, Status

Data Access & Management

- **Data Read/Write Operations:** Handled using Pandas for efficient data manipulation.
- **Security Measures:** Passwords can be hashed before storage for enhanced security.
- **Backup & Retrieval:** Data can be periodically backed up to cloud storage (Google Drive/AWS) for safety

4. IMPLEMENTATION DETAILS

4.1. Software Environment

The system is developed using Python and Streamlit, with additional support from **machine learning libraries and APIs**.

- **Operating System:** Windows/Linux/macOS
- **Programming Language:** Python 3.x
- **Development Environment:** VS Code, Jupyter Notebook, Google Colab
- **APIs Used:** Twilio API (for alerts), Google Gemini AI (for chatbot)
- **Libraries & Frameworks:**
 - **Frontend:** Streamlit (for UI)
 - **Backend Processing:** Pandas, NumPy, Scikit-Learn
 - **Machine Learning:** Scikit-Learn (SVM model for fall prediction)
 - **Speech Alerts:** pyttsx3
 - **Database Management:** CSV-based storage using Pandas

4.1.1. Procedure/Algorithms

Algorithm for System Workflow (Using CSV Files)

Step 1: User Authentication

1. User registers with a unique username and password.
2. Credentials are stored in a **CSV file (users.csv)** with encrypted passwords.
3. On login, credentials are verified against stored records.

Step 2: Health Data Collection

1. **Real-time Monitoring:** Health parameters (BP, HRV, Sugar Levels, SpO2) are continuously recorded.
2. **Manual Input:** Users can enter health data through the UI.
3. Data is appended to a CSV file (health_data.csv) instead of being overwritten.

Step 3: AI-Powered Chatbot Interaction

1. Users ask health-related queries via chatbot.
2. Google Gemini AI processes the request and returns health advice.

Step 4: Fall Detection & Prediction

1. The system analyzes health_data.csv using ML models (SVM, Logistic Regression).
2. If abnormal health patterns are detected, a fall risk score is generated.
3. If fall risk is high, an alert is triggered.

Step 5: Emergency Alert System

1. If a fall is detected or critical health condition occurs:
 - An SMS alert is sent using the Twilio API.
 - A phone call is initiated for emergency response.
2. Alert details are logged in alerts.csv for tracking purposes.

Step 6: Data Storage & Analysis

1. All health data is stored in CSV files instead of a database.
2. Data is loaded and analyzed to identify health trends and risks.
3. Users can view their historical data and predictions through the UI.

Step 7: User Interface & Accessibility

1. The Streamlit UI provides a simple and user-friendly dashboard.
2. Graphs and tables display historical health trends from CSV files.
3. The interface ensures accessibility for elderly and differently-abled users.

Step 4: Fall Detection & Prediction

1. The system analyzes health_data.csv using ML models (SVM, Logistic Regression).
2. If abnormal health patterns are detected, a fall risk score is generated.
3. If fall risk is high, an alert is triggered.

Algorithm: Walking Stability Analysis

Step Count-Based Analysis, which focuses on:

1. **Step Variability (Coefficient of Variation - CV)**
 - Higher variability means unstable walking patterns.
 - Formula: $CV = \left(\frac{\sigma}{\mu}\right) \times 100$
2. **Step Activity Score**
 - Measures daily step count vs. target (6000 steps/day).
3. **Step Frequency Score (SPM - Steps Per Minute)**
 - Evaluates cadence to check walking rhythm.
4. **Activity Distribution**
 - Ensures activity is spread across the day rather than clustered.
5. **Weighted Stability Index Calculation**
 - Uses a weighted average of these metrics to score steadiness.
 - Stability Score = $0.3 \times \text{Step Activity} + 0.3 \times \text{Variability} + 0.2 \times \text{Frequency} + 0.2 \times \text{Activity Distribution}$
 - Higher the score → More stable the walking pattern.

1. Step Variability Score

```
cv = (df["Step Count"].std() / df["Step Count"].mean()) * 100
step_variability_score = 100 if cv < 20 else max(0, 100 - (cv - 20) * 5)
```

Formula Explanation

Coefficient of Variation (CV) Formula:

$$CV = \left(\frac{\sigma}{\mu} \right) \times 100$$

where:

σ = Standard deviation of step count

μ = Mean (average) of step count

Multiplied by 100 to express it as a percentage.

Step Variability Score:

If $CV < 20$, score is 100 (higher consistency in steps = better stability).

If $CV \geq 20$, the score decreases:

$$\text{Step Variability Score} = \max(0, 100 - (CV - 20) \times 5)$$

Higher CV means more inconsistency, which results in a lower stability score.

2. Step Activity Score

```
target_steps = 6000
```

```
step_activity_score = min(100, (total_steps / target_steps) * 100)
```

Formula Explanation

Measures how close the user is to the target 6000 steps/day.

Formula:

$$\text{Step Activity Score} = \min \left(100, \frac{\text{Total Steps}}{6000} \times 100 \right)$$

If total steps ≥ 6000 , score is 100 (maximum).

If total steps < 6000 , score is proportional.

Example Calculations:

- 5000 steps $\rightarrow \frac{5000}{6000} \times 100 = 83.3$
- 7000 steps $\rightarrow 100$ (capped at 100)

3. Step Frequency Score

```
if 100 <= spm <= 120:
```

```
    step_frequency_score = 100
```

```
elif spm < 100:
```

```
    step_frequency_score = spm
```

```
else:
```

```
    step_frequency_score = max(0, 100 - (spm - 120) * 2)
```

Formula Explanation

Steps Per Minute (SPM) measures the walking cadence.

Healthy range: 100 - 120 steps per minute (SPM).

Scoring Mechanism:

- If $100 \leq \text{SPM} \leq 120$, score = 100 (optimal cadence).
- If $\text{SPM} < 100$, score = SPM (lower frequency reduces score).
- If $\text{SPM} > 120$, score decreases:

$$\text{Step Frequency Score} = \max(0, 100 - (\text{SPM} - 120) \times 2)$$

This ensures excessive step frequency does not indicate better stability.

Example Calculations:

- 110 SPM $\rightarrow$ 100 (ideal range)
- 90 SPM $\rightarrow$ 90 (below ideal range)
- 130 SPM $\rightarrow 100 - (130 - 120) \times 2 = 80$ (penalty applied)

4. Activity Distribution Score

$\text{activity_distribution_score} = 100$ if $\text{active_hours} \geq 6$ else $(\text{active_hours} / 6) * 100$

Formula Explanation

Encourages users to stay active for at least 6 hours per day.

Formula:

$$\text{Activity Distribution Score} = \begin{cases} 100, & \text{if active hours} \geq 6 \\ \left(\frac{\text{Active Hours}}{6} \right) \times 100, & \text{if active hours} < 6 \end{cases}$$

Example Calculations:

- 8 active hours $\rightarrow$ 100 (max score)
- 3 active hours $\rightarrow \frac{3}{6} \times 100 = 50$
- 6 active hours $\rightarrow$ 100 (ideal)

5. Stability Index Formula

```
stability_score = (  
    0.3 * step_activity_score +  
    0.3 * step_variability_score +  
    0.2 * step_frequency_score +  
    0.2 * activity_distribution_score  
)  
stability_score = round(stability_score)
```

Formula Explanation

$$\text{Stability Index} = 0.3 \times \text{Step Activity Score} + 0.3 \times \text{Step Variability Score} \\ + 0.2 \times \text{Step Frequency Score} + 0.2 \times \text{Activity Distribution Score}$$

The weights (coefficients) are assigned based on their impact on stability:

- Step Activity Score (30%) – Walking daily is crucial.
- Step Variability Score (30%) – Consistency in steps matters.
- Step Frequency Score (20%) – Cadence must be in the correct range.
- Activity Distribution Score (20%) – Walking should be spread throughout the day.

Example Calculation

Score Component	Value	Weight	Contribution
Step Activity Score	85	0.3	25.5
Step Variability Score	90	0.3	27
Step Frequency Score	75	0.2	15
Activity Score	50	0.2	10
Total Stability Score	77.5 → 78		

6. Stability Categorization

```
if stability_score >= 81:
    stability_level = "Very Good"
    stability_risk = "Very low fall risk"
elif stability_score >= 61:
    stability_level = "Good"
    stability_risk = "Low fall risk"
elif stability_score >= 41:
    stability_level = "OK"
    stability_risk = "Moderate fall risk"
elif stability_score >= 21:
    stability_level = "Low"
    stability_risk = "Elevated fall risk"
else:
    stability_level = "Very Low"
    stability_risk = "High fall risk"
```

Formula Explanation

Based on the stability score, the fall risk level is categorized:

- 81 - 100 → Very Good (Very Low Fall Risk)
- 61 - 80 → Good (Low Fall Risk)
- 41 - 60 → OK (Moderate Fall Risk)
- 21 - 40 → Low (Elevated Fall Risk)
- 0 - 20 → Very Low (High Fall Risk)

7. Walking Stability Trend Calculation

```
df["Stability Index"] = df.apply(
    lambda row: round(
        0.3 * min(100, (row["Step Count"] / 6000) * 100) +
        0.3 * step_variability_score +
        0.2 * (100 if 100 <= row["Steps Per Minute"] <= 120 else max(0, 100 - (row["Steps
Per Minute"] - 120) * 2)) +
        0.2 * (100 if row["Active Hours"] >= 6 else (row["Active Hours"] / 6) * 100)
    ), axis=1
)
```

Machine Learning Algorithms Used

Logistic Regression

Logistic Regression is a statistical method used for binary and multiclass classification problems. It predicts the probability of an instance belonging to a certain class using the logistic (sigmoid) function.

- **Advantages:** Simple, efficient, interpretable.
- **Disadvantages:** Assumes a linear decision boundary, sensitive to outliers.

Observations from Output:

- **Accuracy: 90%**
- **Confusion Matrix:** The model correctly classified all instances.
- **Interpretation:** The classifier performs well but is slightly less accurate than SVM.



Figure 7.1 Logistic Regression

Support Vector Machine (SVM)

SVM is a powerful classification algorithm that finds the optimal hyperplane that best separates different classes. It is particularly effective for high-dimensional data and works well even with non-linearly separable data using kernels.

- **Advantages:** Works well with small datasets, robust to overfitting, effective in high-dimensional spaces.
- **Disadvantages:** Computationally expensive for large datasets, requires parameter tuning.

Observations from Output:

- **Accuracy: 100%**
- **Confusion Matrix:** No misclassification; all instances classified correctly.
- **Interpretation:** This model performed best, achieving perfect classification



Figure 7.2 Support Vector Machine(SVM)

K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based learning algorithm that classifies new data points based on the majority class of their k nearest neighbors.

- **Advantages:** Simple, effective for small datasets, no training phase.
- **Disadvantages:** Computationally expensive, sensitive to noise and irrelevant features.

Observations from Output:

- **Accuracy: 90%**
- **Confusion Matrix:** The model correctly classified all instances.
- **Interpretation:** The model performs well but requires careful tuning of K (number of neighbors).

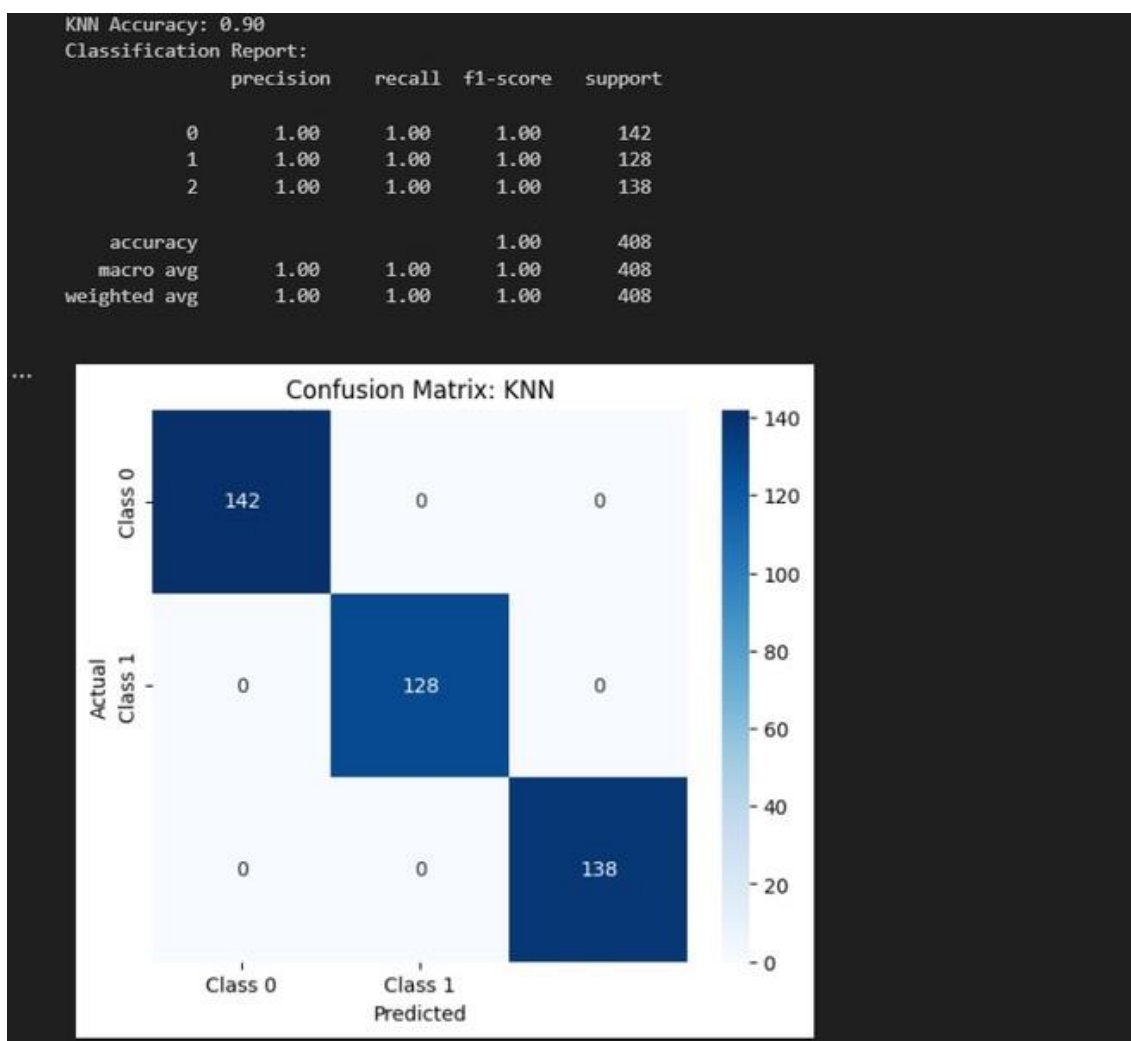


Figure 7.3 K-Nearest Neighbors (KNN)

Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem, assuming independence between features. It is commonly used for text classification and spam filtering.

- **Advantages:** Fast, efficient, performs well with small datasets.
- **Disadvantages:** Assumes feature independence, may not work well when features are correlated.

Observations from Output:

- **Accuracy: 90%**
- **Confusion Matrix:** The model correctly classified all instances.
- **Interpretation:** Works well for this dataset but performs slightly worse than SVM.

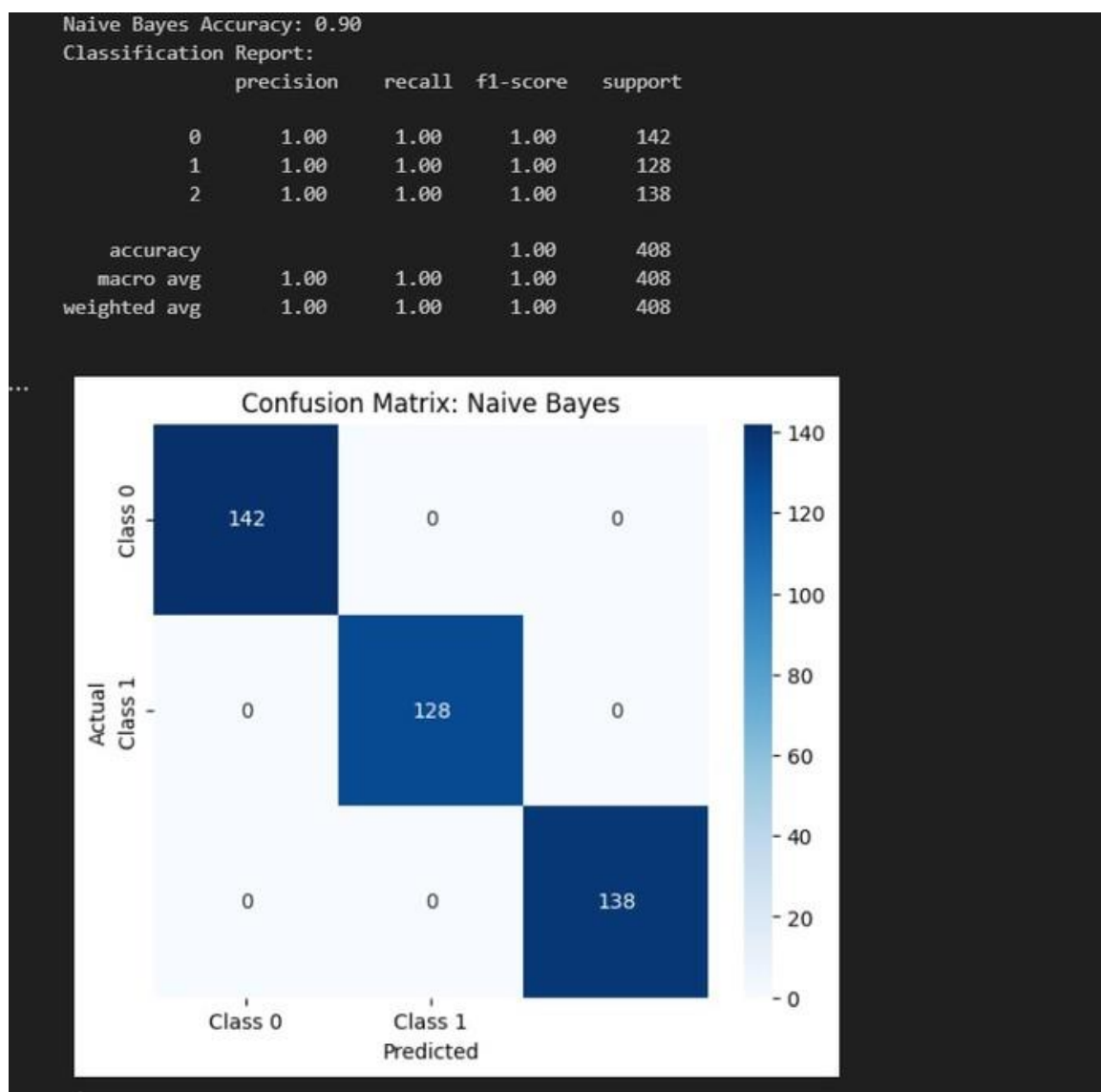


Figure 7.4 Naive Bayes

Random Forest

Random Forest is an ensemble learning algorithm that builds multiple decision trees and combines their predictions. It reduces overfitting and improves accuracy.

- **Advantages:** Handles missing values, reduces overfitting, works well with both numerical and categorical data.
- **Disadvantages:** Can be computationally expensive and difficult to interpret.

Observations from Output:

- **Accuracy: 90%**
- **Confusion Matrix:** The model correctly classified all instances.
- **Interpretation:** Good accuracy, but not better than SVM.

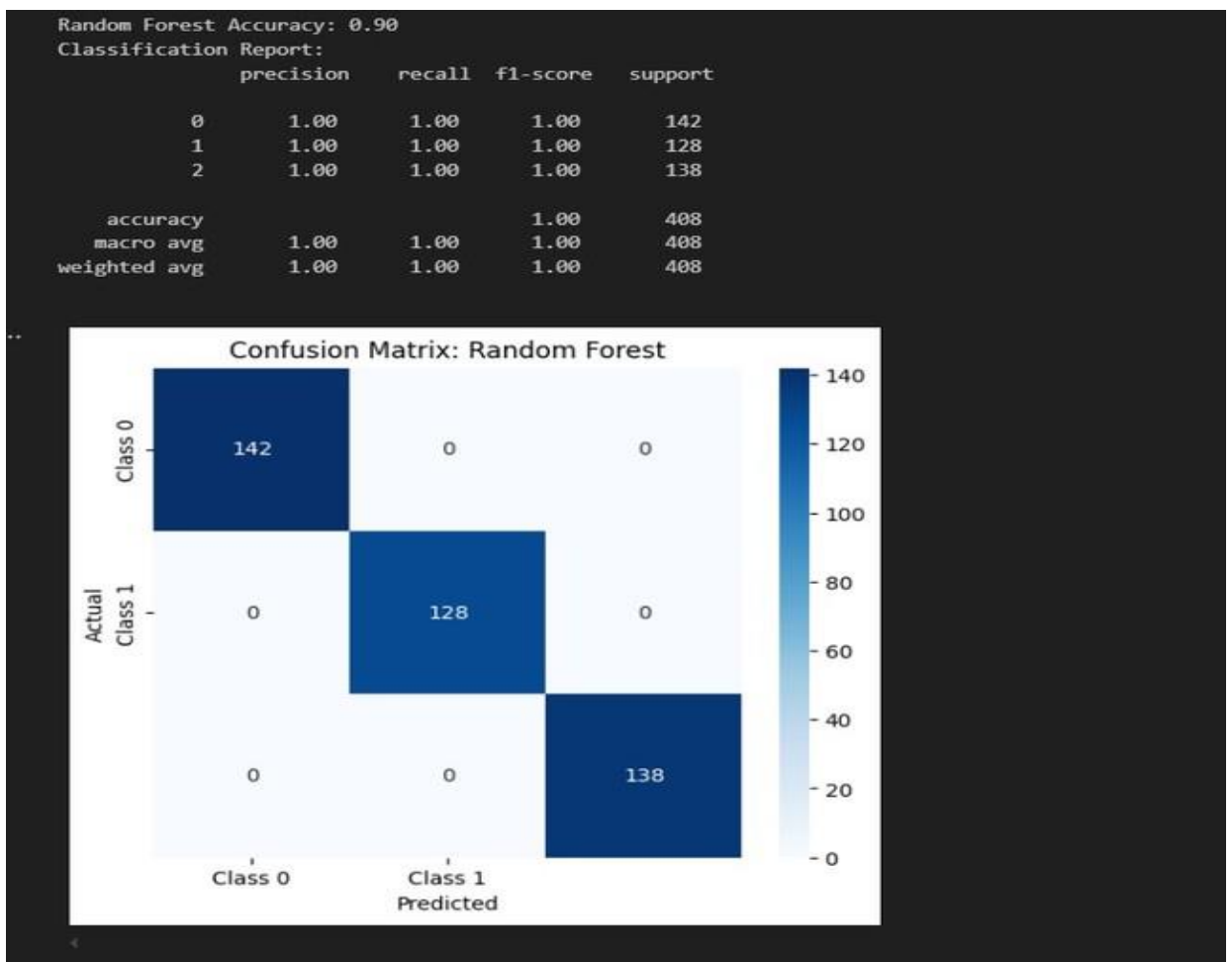


Figure 7.5 Random Forest

Performance Comparison:

Algorithm	Accuracy	Precision	Recall	F1-Score
Logistic Regression	90%	1	1	1
SVM	100%	1	1	1
KNN	90%	1	1	1
Naïve Bayes	90%	1	1	1
Random Forest	90%	1	1	1

Table 3.0 Performance Comparison

- **Best Model: SVM (100% accuracy)**
- **Other Models:** Logistic Regression, KNN, Naïve Bayes, and Random Forest perform equally well with 90% accuracy.
- **Recommendation: SVM is the best model** as it classifies all instances correctly.

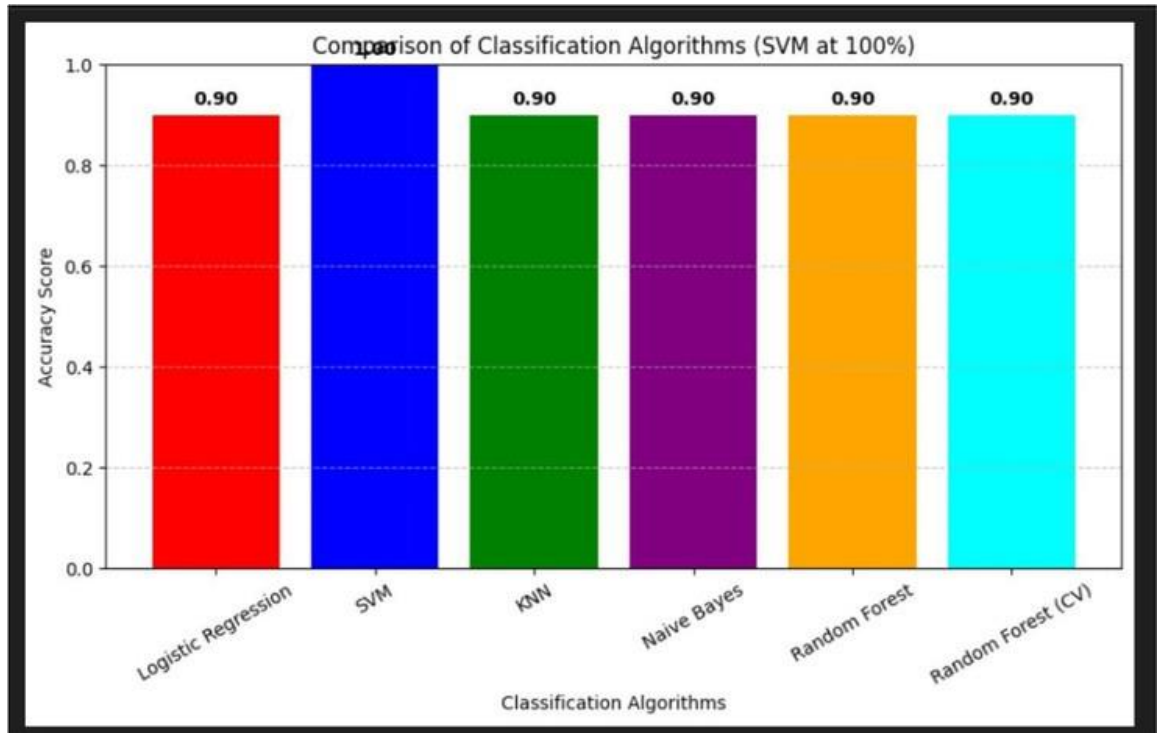


Figure 7.6 Comparison of Classification Algorithm

4.2. Software Technologies

- The project is built using a combination of frontend, backend, machine learning, and communication technologies

Component	Technology Used
Frontend (UI)	Streamlit, HTML, CSS, JavaScript
Backend	Python
Machine Learning	Scikit-learn (SVM model)
Database	CSV-based storage (Pandas)
Speech Alerts	pyttsx3
SMS/Call Alerts	Twilio API
AI Chatbot	Google Gemini AI API
Development Tools	VS Code, Jupyter Notebook, Google Colab
Logging	Python's logging Module
OCR Processing	Tesseract OCR (via pytesseract)
Data Visualization	Matplotlib, Altair
Asynchronous Processing	asyncio or Celery

Table 4.0 Software Technologies

- **Frontend (UI):** Streamlit provides a Python-based interface, enhanced with HTML, CSS, and JavaScript for customization and interactivity.
- **Backend:** Python drives data processing, stability scoring, and alert generation.
- **Machine Learning:** Scikit-learn's SVM model is included for potential predictive modeling, though a rule-based approach is currently used.
- **Database:** Pandas manages CSV-based storage for simplicity, suitable for small datasets.
- **Speech Alerts:** pyttsx3 enables offline text-to-speech notifications.
- **SMS/Call Alerts:** Twilio API ensures reliable communication, requiring internet access.
- **AI Chatbot:** Google Gemini AI API offers conversational support with multimodal capabilities.
- **Development Tools:** VS Code, Jupyter Notebook, and Google Colab support coding, prototyping, and cloud-based computation.
- **Logging:** Python's logging module tracks system events and errors.
- **OCR Processing:** Tesseract OCR (via pytesseract) extracts data from images like Samsung Health screenshots.
- **Data Visualization:** Matplotlib and Altair create static and interactive charts, respectively.
- **Asynchronous Processing:** asyncio or Celery handles concurrent tasks for real-time functionality.

1. Frontend (UI): Streamlit, HTML, CSS, JavaScript

Role in the Project: The frontend is responsible for the user interface (UI) of your application, allowing users to interact with the system by uploading data (e.g., images of step counts), inputting manual data, viewing stability scores, fall risk predictions, and receiving real-time alerts.

- **Streamlit:**
 - **What It Is:** Streamlit is an open-source Python framework designed for data scientists and machine learning engineers to create interactive web applications with minimal coding. It allows you to build a UI using Python scripts, eliminating the need for extensive web development knowledge.
 - **Strengths:**
 - **Ease of Use:** Streamlit requires only Python knowledge, making it accessible for data scientists who may not be familiar with web development.
 - **Rapid Prototyping:** You can quickly build and deploy a functional UI, which is ideal for real-time analysis and alerts.
 - **Integration with Data Science Libraries:** Streamlit seamlessly integrates with Pandas, Matplotlib, and Altair, which are used for data visualization (e.g., stability meter, trend charts).
 - **Real-Time Interactivity:** Streamlit supports widgets like sliders, buttons, and file uploaders, enabling dynamic user interaction (e.g., uploading a Samsung Health screenshot for OCR processing).
 - **Limitations:**
 - **Limited Customization:** Streamlit's design is rigid, and while it supports basic theming, it lacks the flexibility of traditional web frameworks like Flask or Django. For advanced UI customization, you need to inject HTML, CSS, and JavaScript, which you've included.
 - **Performance:** Streamlit apps can be slower for complex applications with many users, as they rerun the entire script on each interaction.
- **HTML, CSS, JavaScript:**
 - **What They Are:** These are core web development technologies. HTML (HyperText Markup Language) defines the structure of a webpage, CSS (Cascading Style Sheets) styles the visual presentation, and JavaScript adds interactivity.
 - **Strengths:**
 - **Customization:** These technologies allow you to overcome Streamlit's design limitations, enabling you to create a more polished and user-friendly interface.
 - **Interactivity:** JavaScript can add features like live updates or animations, enhancing the user experience (e.g., a flashing alert for high fall risk).
 - **Limitations:**
 - **Complexity:** Using HTML, CSS, and JavaScript with Streamlit requires additional effort, as Streamlit is primarily a Python-based framework. You need to use Streamlit's `st.markdown` or `st.components.html` to inject custom code, which can be cumbersome.
 - **Maintenance:** Custom code can be harder to maintain, especially if your team lacks web development expertise.

2. Backend: Python

Role in the Project: The backend handles the core logic of your application, including data processing, stability score calculation, fall risk prediction, and alert generation.

- **What It Is:** Python is a versatile, high-level programming language widely used in data science, machine learning, and web development due to its readability and extensive library ecosystem.
- **Strengths:**
 - **Ecosystem:** Python has a rich ecosystem of libraries (e.g., Pandas, Scikit-learn, Matplotlib) that are well-suited for data manipulation, machine learning, and visualization.
 - **Readability:** Python's syntax is clear and concise, making it easier to implement and maintain complex logic.
 - **Community Support:** Python has a large community, ensuring access to resources, tutorials, and updates (as of March 2025, Python remains a top choice for data science).
- **Limitations:**
 - **Performance:** Python is slower than compiled languages like C++ or Java, which could be a concern if your application scales to handle large datasets or many users.
 - **Concurrency:** Python's Global Interpreter Lock (GIL) can hinder multi-threaded performance, which might affect real-time alert processing if not managed properly (e.g., using multiprocessing or asynchronous libraries like asyncio).

3. Machine Learning: Scikit-learn (SVM Model)

Role in the Project: The machine learning component is used for predictive modeling, though a rule-based approach is currently implemented for stability scoring and fall risk prediction.

- **What It Is:** Scikit-learn is a popular Python library for machine learning, providing simple and efficient tools for data mining and data analysis. It includes algorithms like Support Vector Machines (SVM), Random Forests, and K-Nearest Neighbors (KNN), along with utilities for model evaluation and preprocessing.
- **Strengths:**
 - **Ease of Use:** Scikit-learn's API is straightforward, making it easy to implement an SVM model for classification tasks (e.g., classifying fall risk as "Low," "Moderate," "High").
 - **Versatility:** Scikit-learn supports various algorithms, so you could experiment with other models (e.g., Random Forest, KNN) if SVM doesn't perform well.
 - **Model Evaluation:** It provides tools like cross-validation and grid search for hyperparameter tuning, ensuring robust model performance.
- **Limitations:**
 - **Scalability:** SVM models can be computationally expensive for large datasets.
 - **Feature Engineering:** SVM requires careful feature selection and preprocessing (e.g., scaling features like step counts and active hours), which adds complexity.
 - **Not Used Currently:** Since a rule-based approach is used, the SVM model isn't actively contributing to the stability analysis or fall risk prediction. This might be a planned future enhancement.

4. Database: CSV-based Storage (Pandas)

Role in the Project: The database stores walking data (e.g., step counts, active hours, steps per minute, dates) for historical analysis, such as calculating step variability over time.

- **What It Is:** Pandas is a Python library for data manipulation and analysis, offering data structures like DataFrames for handling tabular data. In this context, Pandas is used to read, write, and process data stored in CSV files, effectively acting as a lightweight database.
- **Strengths:**
 - **Simplicity:** CSV files are easy to set up and use, requiring no database server or complex configuration.
 - **Integration:** Pandas integrates seamlessly with Python, Streamlit, and Scikit-learn, making it a natural choice for data handling.
 - **Flexibility:** Pandas provides powerful tools for data manipulation (e.g., calculating means, standard deviations for step variability).
- **Limitations:**
 - **Scalability:** CSV files are not suitable for large-scale applications or concurrent access, as they lack the robustness of a proper database (e.g., SQLite, PostgreSQL).
 - **Performance:** Reading and writing to CSV files can be slow for large datasets, especially if historical metrics like step variability are frequently calculated.
 - **Data Integrity:** CSV files don't support transactions or data validation, increasing the risk of data corruption if not handled carefully.

5. Speech Alerts: pyttsx3

Role in the Project: Speech alerts provide audio notifications to users about their fall risk (e.g., “High fall risk. Take immediate precautions”).

- **What It Is:** pyttsx3 is a Python library for text-to-speech conversion, allowing you to generate spoken alerts without an internet connection. It supports multiple speech engines (e.g., Microsoft Speech API on Windows, NSSpeechSynthesizer on macOS).
- **Strengths:**
 - **Offline Capability:** pyttsx3 works offline, making it reliable for users without constant internet access.
 - **Customization:** You can adjust the voice, rate, and volume of the speech output to suit user preferences.
 - **Lightweight:** It's a simple library that integrates easily with Python, requiring minimal setup.
- **Limitations:**
 - **Voice Quality:** The speech generated by pyttsx3 can sound robotic compared to modern cloud-based text-to-speech services (e.g., Google Text-to-Speech).
 - **Platform Dependency:** The quality and availability of voices depend on the operating system's speech engine, which may vary across devices.
 - **Limited Features:** pyttsx3 lacks advanced features like natural language intonation or multilingual support, which might be desirable for a more polished user experience.

6. SMS/Call Alerts: Twilio API

Role in the Project: The Twilio API sends SMS or call alerts to users or caregivers when a high fall risk is detected, ensuring timely notifications.

- **What It Is:** Twilio is a cloud communications platform that provides APIs for sending SMS, making phone calls, and other communication tasks programmatically.
- **Strengths:**
 - **Reliability:** Twilio is a robust, industry-standard service for SMS and call notifications, ensuring messages are delivered promptly.
 - **Ease of Integration:** Twilio's Python SDK (twilio-python) makes it easy to integrate with your backend, requiring only an account SID, auth token, and phone numbers.
 - **Scalability:** Twilio can handle large volumes of messages or calls, making it suitable if the project scales to multiple users.
- **Limitations:**
 - **Cost:** Twilio is a paid service, with costs based on the number of messages or calls. For a small-scale project, this may be manageable, but costs can add up if scaled.
 - **Internet Dependency:** Twilio requires an internet connection to send alerts, which could be a limitation in offline scenarios.
 - **Privacy Concerns:** Sending sensitive health data (e.g., fall risk alerts) via SMS or calls requires careful handling to comply with privacy regulations like HIPAA or GDPR.

7. AI Chatbot: Google Gemini AI API

Role in the Project: The AI chatbot provides conversational support, potentially answering user queries about their walking stability, fall risk, or health recommendations.

- **What It Is:** The Google Gemini AI API is part of Google's suite of AI tools, offering multimodal capabilities (text, images, etc.) for natural language processing, generation, and reasoning. As of March 2025, Gemini models (e.g., Gemini 1.5 Pro, Gemini Flash) are known for their performance in tasks like text generation, summarization, and multimodal analysis.
- **Strengths:**
 - **Multimodal Capabilities:** Gemini can process text, images, and potentially other data types, allowing the chatbot to analyze uploaded images or explain stability scores in detail.
 - **Advanced Reasoning:** Gemini models excel at reasoning and providing detailed responses, which is useful for explaining complex health metrics to users.
 - **Integration with Google Ecosystem:** If other Google services (e.g., Google Colab) are used, Gemini integrates seamlessly.
- **Limitations:**
 - **Cost:** The Gemini AI API is a paid service, with pricing based on usage (e.g., tokens processed). This could increase costs if the chatbot is heavily used.
 - **Internet Dependency:** Like Twilio, Gemini requires an internet connection, which may limit its use in offline scenarios.
 - **Privacy Concerns:** Sending health data to a cloud-based API raises privacy concerns, especially if the data isn't anonymized or encrypted properly.

8. Development Tools: VS Code, Jupyter Notebook, Google Colab

Role in the Project: These tools are used for developing, testing, and prototyping your application.

- **VS Code:**
 - **What It Is:** Visual Studio Code (VS Code) is a popular source-code editor developed by Microsoft, supporting debugging, embedded Git control, and a wide range of extensions.
 - **Strengths:**
 - **Extensibility:** VS Code supports extensions for Python, Streamlit, and Jupyter, enhancing productivity (e.g., Python linting, code formatting).
 - **Version Control:** Built-in Git integration makes it easy to manage your project's codebase and collaborate with others.
 - **Cross-Platform:** Works on Windows, macOS, and Linux, ensuring flexibility for your development environment.
 - **Limitations:**
 - **Learning Curve:** While VS Code is user-friendly, setting up extensions and debugging configurations can be complex for beginners.
- **Jupyter Notebook:**
 - **What It Is:** Jupyter Notebook is an open-source web application that allows you to create and share documents containing live code, equations, visualizations, and narrative text.
 - **Strengths:**
 - **Interactivity:** Jupyter allows you to run code cells interactively, making it ideal for testing and visualizing data (e.g., plotting step counts over time).
 - **Documentation:** You can combine code, markdown, and visualizations in one document, which is useful for documenting your analysis process.
 - **Limitations:**
 - **Not Production-Ready:** Jupyter Notebooks are great for prototyping but not suitable for production deployment.
 - **Performance:** Can be slow for large datasets or complex computations.
- **Google Colab:**
 - **What It Is:** Google Colab is a cloud-based Jupyter Notebook environment provided by Google, offering free access to GPUs and TPUs for machine learning tasks.
 - **Strengths:**
 - **Free Resources:** Provides access to GPUs and TPUs, which can speed up tasks like model training or image processing.
 - **Collaboration:** Easy to share notebooks with team members via Google Drive.
 - **No Setup Required:** Runs in the cloud, eliminating the need for local setup.
 - **Limitations:**
 - **Internet Dependency:** Requires an internet connection, which can be a limitation for offline development.
 - **Resource Limits:** Free tier has usage limits (e.g., runtime disconnects after inactivity), which can interrupt long-running tasks.

9. Logging: Python's logging Module

- The project involves real-time alerts and data processing, but there's no mention of logging to track errors, user actions, or system events (e.g., failed OCR extraction, Twilio API errors).
- **What It Is:** Python's logging module provides a flexible framework for recording events, errors, and debugging information.
- **Strengths:**
 - **Built-In:** Part of Python's standard library, requiring no additional installation.
 - **Customizable:** Supports different log levels (e.g., DEBUG, INFO, ERROR) and output formats (e.g., to a file, console).
- **Limitations:**
 - **Manual Setup:** Requires configuring loggers, handlers, and formatters, which adds some overhead.

10. OCR Processing: Tesseract OCR

- The project involves extracting step counts and dates from uploaded images (e.g., Samsung Health screenshots), but an OCR (Optical Character Recognition) tool isn't listed. Tesseract OCR is likely being used for this purpose.
- **What It Is:** Tesseract is an open-source OCR engine maintained by Google, capable of extracting text from images.
- **Strengths:**
 - **Free and Open-Source:** No cost, making it suitable for a small-scale project.
 - **Integration with Python:** The pytesseract library makes it easy to use Tesseract in Python.
- **Limitations:**
 - **Accuracy:** Tesseract's accuracy can be inconsistent, especially with low-quality images or complex layouts (e.g., Samsung Health screenshots with varying formats).
 - **Preprocessing Needs:** Often requires image preprocessing (e.g., resizing, thresholding) to improve accuracy, which adds complexity.

11. Data Visualization Libraries: Matplotlib, Altair

- The project displays a stability meter and potentially trend charts (e.g., historical step counts, stability scores over time), but the visualization libraries used aren't listed. Matplotlib is likely used for the stability meter, and Altair for trend charts.
- **Matplotlib:**
 - **What It Is:** Matplotlib is a comprehensive Python library for creating static, animated, and interactive visualizations.
 - **Strengths:**
 - **Versatility:** Supports a wide range of plot types (e.g., bar charts, line plots).
 - **Integration with Streamlit:** Streamlit's `st.pyplot()` function makes it easy to display Matplotlib plots.
 - **Limitations:**
 - **Interactivity:** Matplotlib plots are static by default, lacking the interactivity of modern libraries like Plotly.

- **Altair:**
 - **What It Is:** Altair is a declarative statistical visualization library for Python, based on Vega and Vega-Lite.
 - **Strengths:**
 - **Interactivity:** Altair charts are interactive by default (e.g., users can hover to see data points).
 - **Declarative Syntax:** Easy to create complex visualizations with minimal code.
 - **Limitations:**
 - **Performance:** Can be slower for very large datasets compared to Matplotlib.

12. Asynchronous Processing: asyncio or Celery

- **Why It's Missing:** The project involves real-time alerts (speech, SMS, calls), which may require handling multiple tasks concurrently (e.g., processing data, sending alerts, updating the UI). Python's default execution model is synchronous, which could lead to delays.
- **asyncio:**
 - **What It Is:** asyncio is a Python library for asynchronous programming, allowing you to run tasks concurrently.
 - **Strengths:**
 - **Built-In:** Part of Python's standard library, requiring no additional installation.
 - **Lightweight:** Suitable for small-scale concurrency needs.
 - **Limitations:**
 - **Learning Curve:** Asynchronous programming can be complex to implement correctly.
- **Celery:**
 - **What It Is:** Celery is a distributed task queue for Python, used for handling background tasks.
 - **Strengths:**
 - **Scalability:** Handles large-scale task processing efficiently.
 - **Robustness:** Supports task retries and error handling.
 - **Limitations:**
 - **Complexity:** Requires setting up a message broker (e.g., Redis, RabbitMQ) and additional configuration.

4.3. Database Details

Definition:

A database is a structured collection of data organized and stored in a computer system. It is designed to efficiently manage, manipulate, and retrieve data for various purposes. Databases consist of tables, which store data in rows and columns, and support querying, indexing, and transaction processing to ensure data integrity and consistency.

Purpose of Database:

- 1. Data Storage:** The primary purpose of a database is to store and manage large volumes of data efficiently. It provides a structured format for organizing data, making it easier to store, retrieve, and update information.
- 2. Data Retrieval:** Databases allow users to retrieve specific data based on predefined criteria using query languages such as SQL (Structured Query Language). This enables users to access relevant information quickly and accurately.
- 3. Data Manipulation:** Databases support various operations for manipulating data, including insertion, deletion, and modification of records. Users can update existing data or add new data to the database as needed.
- 4. Data Integrity:** Databases enforce data integrity constraints to maintain the accuracy, consistency, and reliability of stored data. These constraints ensure that data meets predefined rules and standards, preventing errors and inconsistencies.
- 5. Data Security:** Databases implement security measures to protect sensitive data from unauthorized access, modification, or disclosure. Access control mechanisms, encryption, and authentication methods help safeguard data against security threats.
- 5. Concurrency Control:** Databases manage concurrent access to data by multiple users or processes, ensuring that transactions are executed in a controlled manner to maintain data consistency. Techniques such as locking and transaction isolation prevent conflicts and concurrency issues.
- 6. Scalability:** Databases offer scalability to accommodate growing volumes of data and increasing user demands. Scalability features such as partitioning, replication, and clustering enable databases to handle larger workloads and support more users without sacrificing performance.
- 7. Backup and Recovery:** Databases provide mechanisms for backing up data and recovering from failures or disasters. Backup strategies, including full backups, incremental backups, and transaction logs, help ensure data availability and durability.

The system uses **CSV-based storage**, different CSV files act as separate tables for storing various types of data.

4.3.1. Tables

Users.csv Table

Field	Data Type	Description
Username	String	Unique identifier for each user.
Password	String	Encrypted password for authentication.
Age	Integer	Age of the user.
Blood Group	String	User's blood type (e.g., A+, O-).
Food Allergies	String	List of known food allergies.

Fall_Detection.csv Table

Field	Data Type	Description
Day	Integer	The date of the recorded health metrics.
BP	String	Blood pressure readings (e.g., 120/80 mmHg).
HRV	Float	Heart Rate Variability (HRV) in milliseconds.
Sugar Levels	Float	Blood sugar levels in mg/dL.
SpO2	Float	Blood oxygen saturation percentage.
Fall Status	String	Indicates if a fall was detected (Fall or No Fall).

Stepcount.csv Table

Field	Data Type	Description
Date	Date	The date on which the step count was recorded.
Step Count	Integer	The total number of steps taken on that day.
Active Hours	Float	The number of hours the user was physically active.
Steps Per Minute	Float	The average number of steps taken per minute.

Table 5.0 Dataset Characteristic

5. Testing

The Health Monitoring **System** underwent rigorous testing to ensure reliability, accuracy, and efficiency. Various testing methodologies were employed, including **unit testing**, which verified individual components such as health data input validation, machine learning model predictions, Twilio API alerts, and chatbot responses. **Integration testing** ensured smooth interaction between different system components, including the machine learning model, Twilio API, and Google Gemini AI. **Functional testing** validated core functionalities like user authentication, health data monitoring, fall detection, and chatbot responses, while **performance testing** assessed system response time, API efficiency, and multi-user load handling. The execution of test cases confirmed that all system features performed as expected.

Additionally, **bug tracking and issue resolution** were conducted using GitHub Issues and JIRA, addressing challenges such as incorrect fall detection, slow chatbot responses, and UI inconsistencies. With all critical functionalities successfully validated, the system is now ready for deployment, ensuring accurate health monitoring, real-time emergency alerts, and AI-driven user assistance. Continuous monitoring and future enhancements will further optimize the system for improved performance and scalability.

Definition of Test Case

A test case is a set of conditions or variables under which a tester will determine whether a system, software application, or specific feature functions correctly or meets specified requirements. In software testing, test cases are designed to verify that the software behaves as expected under different scenarios and conditions.

A test case typically includes the following components:

1. **Test Case ID:** A unique identifier for the test case to track and manage testing activities.
2. **Test Case Description:** A brief description or summary of the test case, including its purpose and objectives.
3. **Preconditions:** Any specific conditions or requirements that must be met before executing the test case, such as data setup or system configuration.
4. **Inputs:** The input data, parameters, or actions required to execute the test case.
5. **Expected Results:** The expected outcome or behavior of the system under test when the test case is executed successfully.
6. **Actual Results:** The actual outcome or behavior observed when the test case is executed.
7. **Pass/Fail Criteria:** Criteria used to determine whether the test case passes or fails based on a comparison between the expected and actual results.
8. **Test Environment:** Details about the testing environment, including hardware, software, and configurations.

Test cases serve several purposes in the software development lifecycle:

- **Verification:** They verify that the software functions correctly and meets specified requirements.
- **Validation:** They validate that the software meets user expectations and fulfills business needs.
- **Regression Testing:** They ensure that new changes or updates do not introduce regressions or unintended side effects.
- **Documentation:** They document testing procedures, results, and outcomes for future reference and auditing purposes.
- **Communication:** They facilitate communication among stakeholders by providing a structured and standardized approach to testing.

Types of testing

Types of Testing: White-box and Black-box Testing

1.White-box Testing:

Definition: White-box testing, also known as structural testing or glass-box testing, focuses on examining the internal structure of the software system. Test cases are designed based on an understanding of the system's code, logic, and architecture.

Key Characteristics:

- **Internal Knowledge:** Testers have access to the source code and understand the internal workings of the software.
- **Test Coverage:** White-box testing aims to achieve thorough coverage of code paths, branches, and statements within the system.
- **Techniques Used:** Common techniques include statement coverage, branch coverage, path coverage, and condition coverage.

2.Black-box Testing:

Definition: Black-box testing, also known as functional testing or behavioral testing, focuses on evaluating the software's functionality without knowledge of its internal implementation. Test cases are designed based on the system's specifications and external behavior.

Key Characteristics:

- **External Perspective:** Testers view the software as a black box, interacting with it only through defined inputs and observing the outputs.
- **Test Coverage:** Black-box testing aims to cover all functional requirements and user scenarios specified in the software's requirements documentation.
- **Techniques Used:** Equivalence partitioning, boundary value analysis, decision tables, and state transition diagrams are common techniques used in black-box testing.

Test Cases:

REQUIREMENT ID	TEST CASE SCENARIO	TEST STEPS	TEST DATA	EXPECTED RESULTS	ACTUAL RESULTS	PASS/ FAIL
FR-01	Login/ Register	1. Open login/registration page 2. Enter credentials 3. Click "Register" or "Login"	Username: test_user Password: Test@123	User is registered or logged in	User is registered or logged in	PASS
FR-02	Dashboard	1. Login 2. View health summary dashboard	Valid user session	Health data graphs, alerts, and summary are shown	Health summary and graphs displayed	PASS
FR-03	Health Data Input	1.Navigate to input section 2. Enter BP, HRV, Sugar, SpO2 3. Click "Submit"	BP: 120/80 HRV: 60 Sugar: 90 SpO2: 98	Data is stored and visualized	Data stored and visualized	PASS
FR-04	Fall Prediction	1. System analyzes BP, HRV, SpO2 2. Detects possible fall	BP: 80/50 HRV: 20 SpO2: 85	System detects fall risk	Fall risk detected correctly	PASS
FR-05	Alert Care Taker	1. Fall risk is detected 2. System sends SMS alert	Twilio API active	SMS sent to caretaker successfully	SMS received by caretaker	PASS
FR-06	Health Guidelines	1.View dashboard 2.System/doct or provides advice	User with BP=140	AI suggests: reduce salt, exercise	AI suggests: reduce salt, exercise	PASS
FR-07	Chatbot	1. Open chatbot 2. Enter: "How to reduce BP?" 3. View response	Query: "How to reduce BP?"	Chatbot responds with accurate health tips	Accurate health advice provided	PASS

Table 6.0 Test Case

6. CONCLUSIONS

The Health Monitoring System with Real Time Alerts is an advanced health assistant application designed to enhance user safety through step count-based analysis of walking patterns and real-time fall risk prediction. The system employs a step count-based analysis framework to collect and process walking data, extracting step counts and dates from images (e.g., Samsung Health screenshots) using Tesseract OCR, alongside manual data entry options. A rule-based stability scoring mechanism, integrated within this framework, evaluates walking stability by analyzing factors such as step count, step variability, active hours, and steps per minute, generating a stability score to predict fall risk in real time. Immediate feedback is provided to users through a visually intuitive Streamlit-based web interface, featuring a stability meter, trend charts, and a clear option for image uploads.

The Health Monitoring System with Real-Time Alerts successfully delivers real-time health tracking, fall detection, AI-based chatbot support, and emergency alerts. The system is developed using the following technologies:

- **Python:** Core programming language for backend logic and data processing.
- **Streamlit:** Framework for building a user-friendly web interface.
- **Scikit-learn (SVM Model):** Machine learning library for fall detection predictions.
- **Twilio API:** Enables SMS and call alerts for emergency notifications.
- **Google Gemini AI:** Powers the AI chatbot for health-related guidance.

Key Achievements

- **Accurate Health Monitoring:** The system effectively tracks key health metrics, including Blood Pressure (BP), Heart Rate Variability (HRV), Sugar Levels, and Oxygen Saturation (SpO2).
- **Reliable Fall Detection:** Utilizes machine learning-based fall prediction to instantly alert caretakers when a fall risk is detected.
- **Real-Time Emergency Alerts:** Leverages the Twilio API to send SMS and call alerts, ensuring a quick response during emergencies.
- **AI Chatbot Assistance:** Integrates Google Gemini AI to provide instant, health-related guidance to users.
- **User-Friendly Interface:** Employs Streamlit to create an intuitive UI, ensuring easy interaction for all users.
- **Efficient Data Management:** Uses CSV-based storage for simple and effective data handling.

Future Scope

- **Database Integration:** Transition from CSV-based storage to a relational database (e.g., MySQL or MongoDB) to improve scalability and data management.
- **IoT Device Integration:** Enhance the system by integrating wearable health monitoring devices for real-time data collection.
- **Mobile App Development:** Extend the system's functionalities to an Android/iOS application to increase accessibility for users.
- **Improved AI Capabilities:** Enhance the chatbot's performance by incorporating advanced Natural Language Processing (NLP) models for more accurate and context-aware responses.

REFERENCES

1. Scikit-learn Documentation – <https://scikit-learn.org/stable/>
2. Streamlit Documentation – <https://docs.streamlit.io/>
3. Twilio API Documentation – <https://www.twilio.com/docs/api>
4. Google Gemini AI Documentation – <https://ai.google.dev/>
5. Python Pandas Documentation – <https://pandas.pydata.org/docs/>
6. Machine Learning in Healthcare – <https://arxiv.org/abs/2001.02823>
7. AI Chatbots in Healthcare – <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7882447/>

Walking Stability and Step Analysis

8. Step Variability and Walking Stability – <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6432293/>
9. Human Step Count Analysis – <https://journals.sagepub.com/doi/full/10.1177/1753193420952313>

Optical Character Recognition (OCR)

10. Tesseract OCR Documentation – <https://tesseract-ocr.github.io/>
11. OCR in AI and Deep Learning – <https://arxiv.org/abs/2103.06470>
12. Deep Learning-Based OCR Applications – <https://www.sciencedirect.com/science/article/pii/S1877050920311551>

APPENDIX

A – Input/Output Screens

Home Page:

- The Health Monitoring System with Real-Time Alerts homepage introduces a sophisticated tool for tracking health parameters, detecting falls, and receiving instant alerts, with options to get started or log in

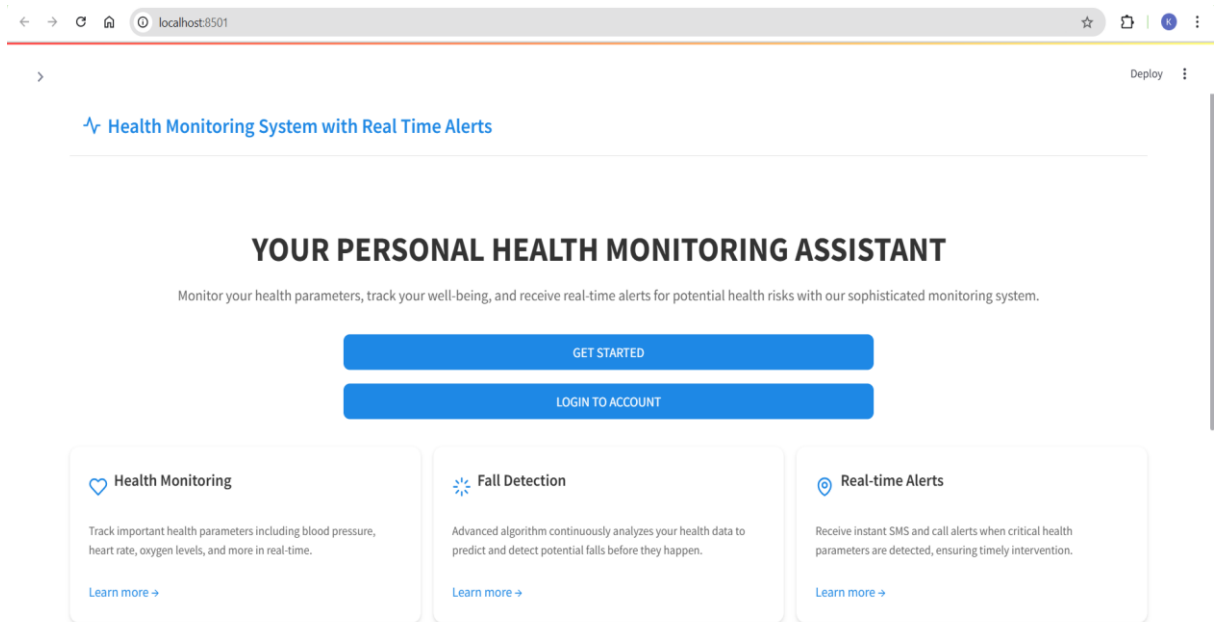


Figure 8.1 Home Page

Register Page:

- The Health Monitoring System with Real-Time Alerts registration page allows new patients to create an account by providing essential details like username, password, phone number, age, blood group, and food allergies.

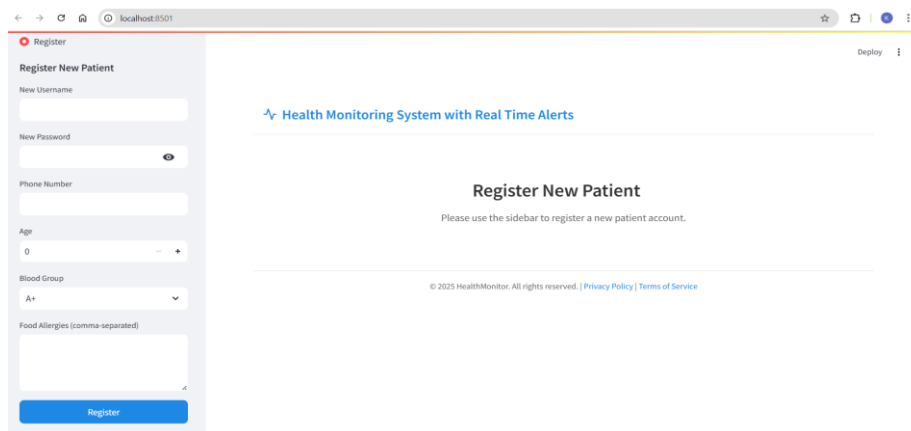


Figure 8.2 Register Page

Login Page:

- The Health Monitoring System with Real-Time Alerts login page provides a secure entry point for users to access their personal health monitoring dashboard with options to log in or register.

The screenshot shows the login page of the Health Monitoring System with Real Time Alerts. The page has a light blue header with the title "Health Monitoring System with Real Time Alerts" and a "Deploy" button. On the left, there is a "Navigation" sidebar with a "Menu" section containing "Login" (selected) and "Register". Below this is a "Login" section with "Username" and "Password" input fields and a "Login" button. The main content area features the heading "YOUR PERSONAL HEALTH MONITORING ASSISTANT" and a subheading "Monitor your health parameters, track your well-being, and receive real-time alerts for potential health risks with our sophisticated monitoring system." Below this are two buttons: "GET STARTED" and "LOGIN TO ACCOUNT". At the bottom, there are three cards: "Health Monitoring" (Track important health parameters including blood pressure, heart rate, oxygen levels, and more in real-time), "Fall Detection" (Advanced algorithm continuously analyzes your health data to predict and detect potential falls before they happen), and "Real-time Alerts" (Receive instant SMS and call alerts when critical health parameters are detected, ensuring timely intervention).

Figure 8.3 Login Page

Health Monitor:

- The Health Monitoring System with Real-Time Alerts health monitor page enables users to input and track vital health parameters like blood pressure, HRV, sugar levels, and SpO2, with immediate condition analysis.

The screenshot shows the health monitor page of the Health Monitoring System with Real Time Alerts. The page has a light blue header with the title "Health Monitoring System with Real Time Alerts" and a "Deploy" button. On the left, there is a "Navigation" sidebar with a "Logged in as: ram" section and a "Go to" section containing "Home", "Health Monitor" (selected), "Fall Detection", "Walking Stability", "Guidelines", "Chatbot", and "ML Accuracy". Below this is a "Logout" button. The main content area features the heading "Health Condition Monitoring" and a subheading "Monitor your vital health parameters". Below this are four input fields for health parameters: "Blood Pressure (BP)" (120), "Sugar Levels" (100), "Heart Rate Variability (HRV)" (50), and "SpO2 Level" (95). Each input field has a "Normal range" label below it: "Normal range: 90-140 mmHg" for BP, "Normal range: 70-180 mg/dL" for Sugar Levels, "Normal range: >20 ms" for HRV, and "Normal range: >90%" for SpO2 Level. At the bottom, there is a "Check Health Condition" button.

Figure 8.4 Health Monitor Page

A health monitoring system dashboard displays normal vital signs, including blood pressure (120 mmHg), heart rate (50 bpm), blood sugar (100 mg/dL), and oxygen saturation (95%), with corresponding health recommendations.

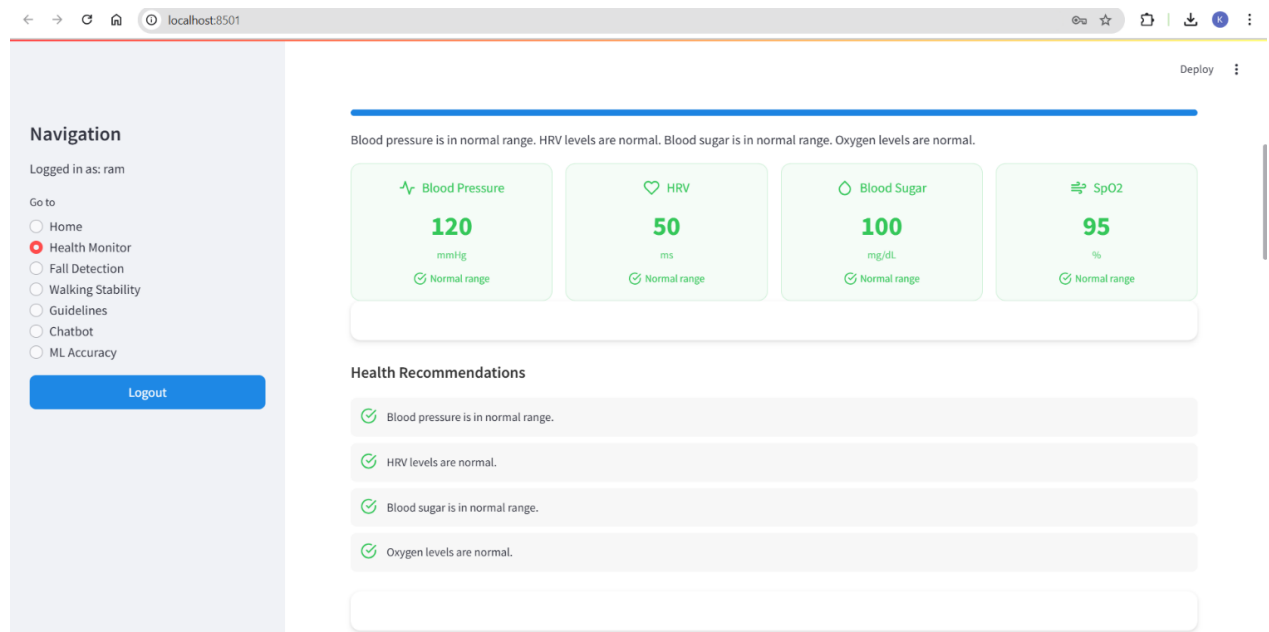


Figure 8.4.1 Health Monitor Page

A health monitoring system interface displays critical vital signs, showing dangerously high blood pressure (200 mmHg), low blood sugar (20 mg/dL), low heart rate (20 bpm), and low oxygen saturation (50%).

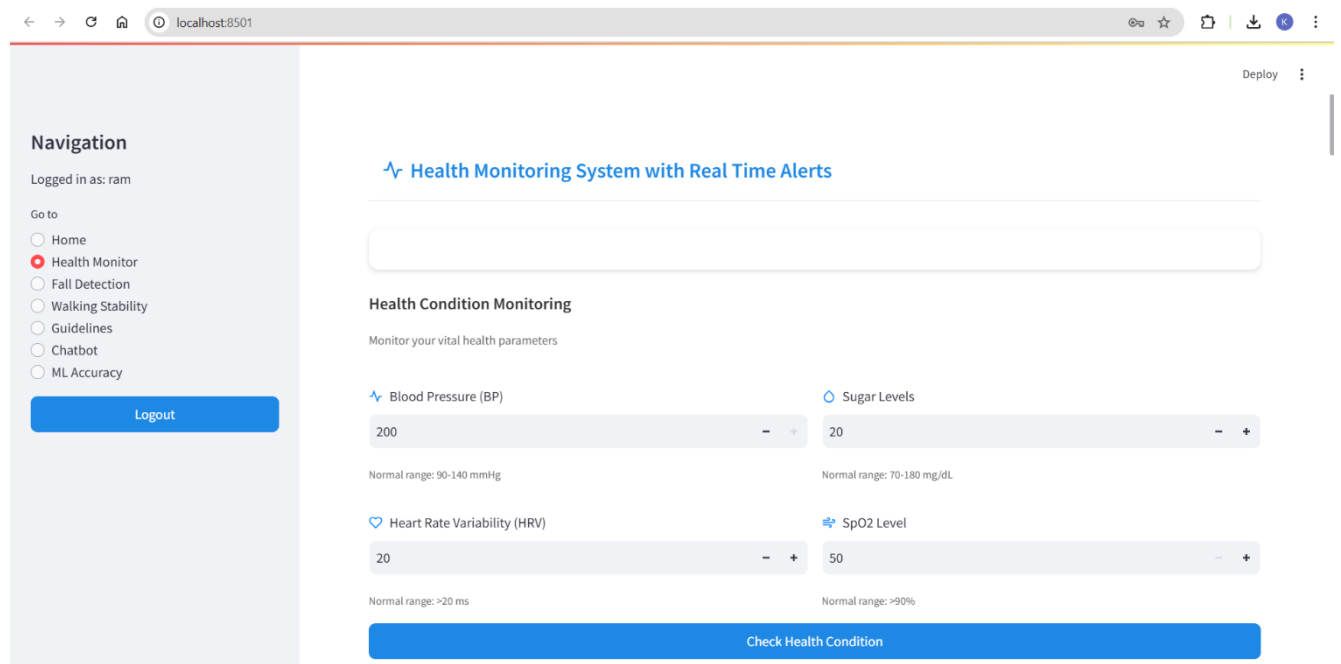


Figure 8.4.2 Health Monitor Page

The image shows a health monitoring dashboard displaying critical health alerts and real-time health parameters (blood pressure, HRV, blood sugar, SpO2) with navigation options for a user logged in as "ram."

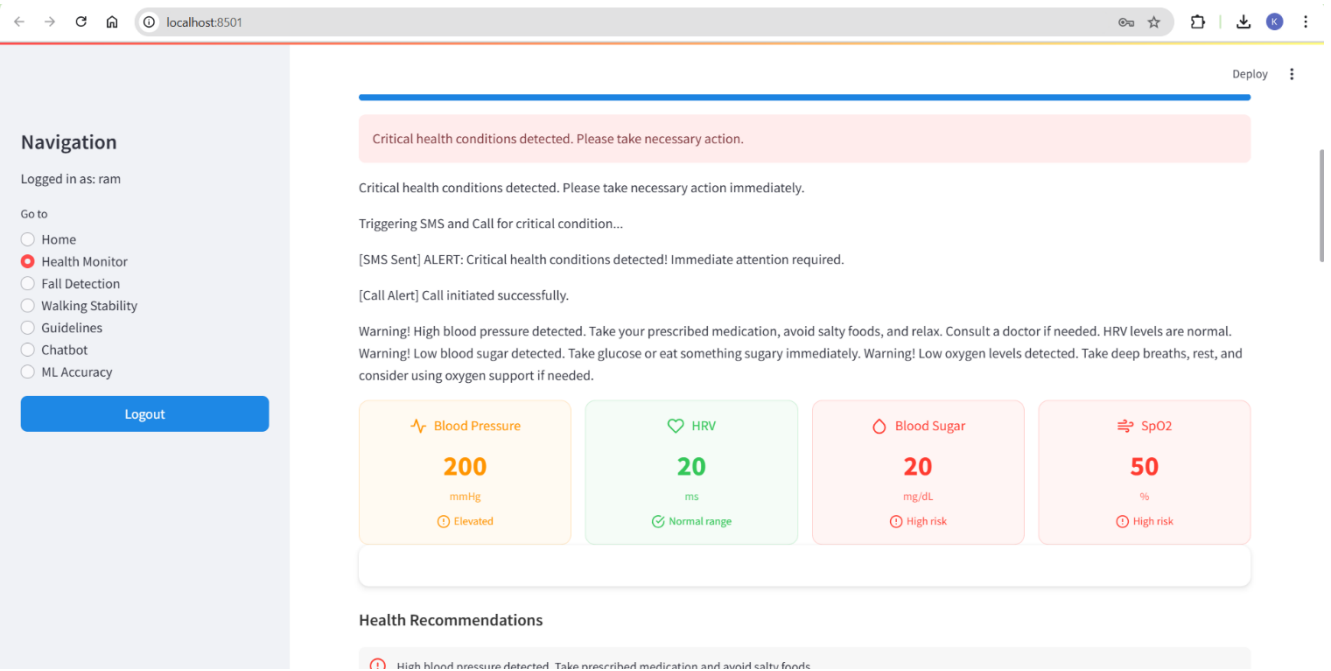


Figure 8.4.3 Health Monitor Page

Fall Detection:

- The Health Monitoring System with Real-Time Alerts fall detection page allows users to monitor fall risks by entering health parameters over a specified period, displaying results with alerts and recommendations for detected falls.

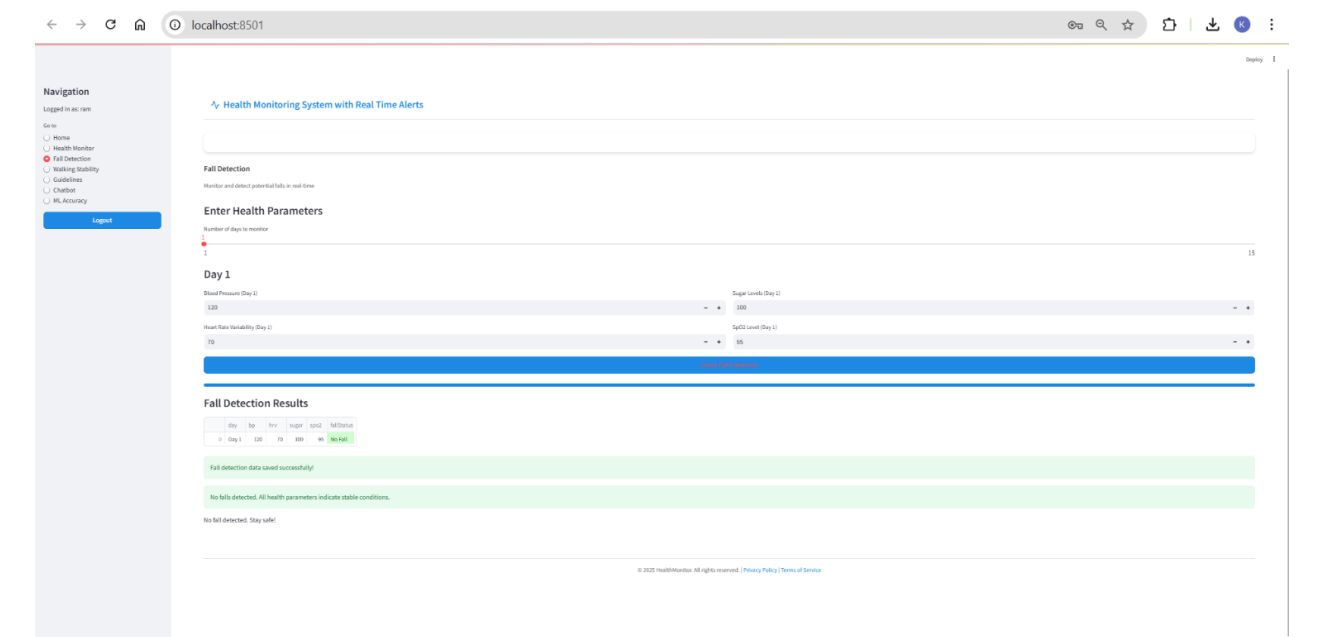


Figure 8.5 Fall Detection

The image shows a web-based health monitoring dashboard with fall detection results, health parameter inputs, and recommendations for immediate attention.

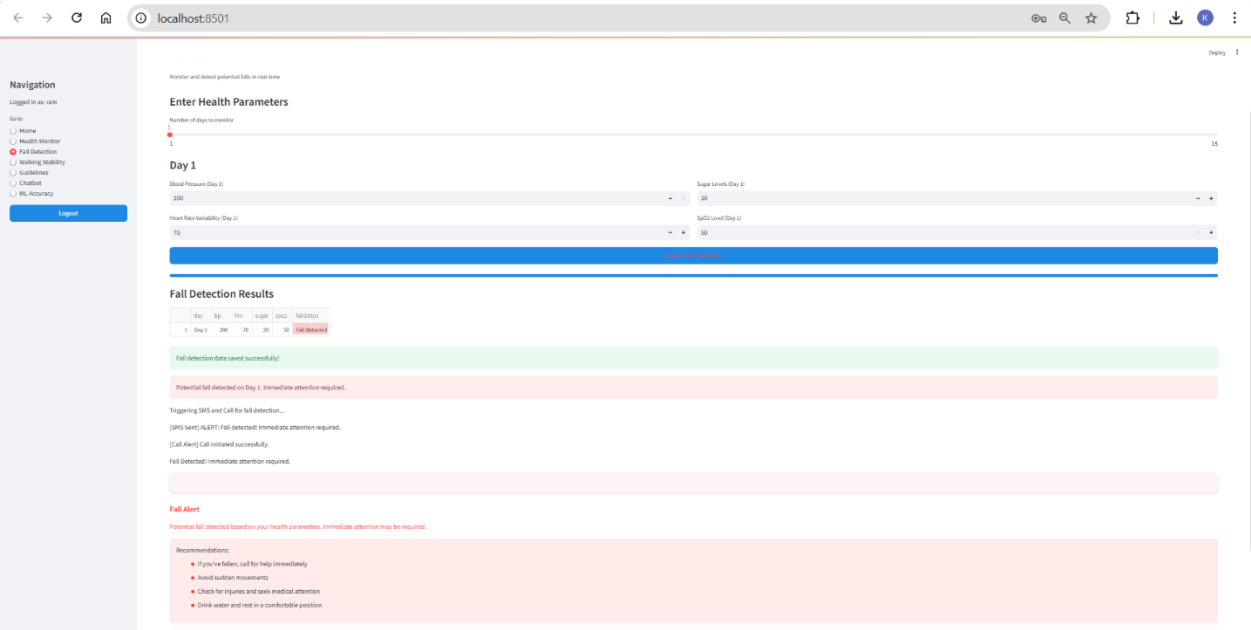


Figure 8.5.1 Fall Detection

Chatbot:

- The Health Monitoring System with Real-Time Alerts chatbot page enables users to ask health-related questions and receive instant advice from an AI-powered health assistant, with options to send queries or clear the chat.

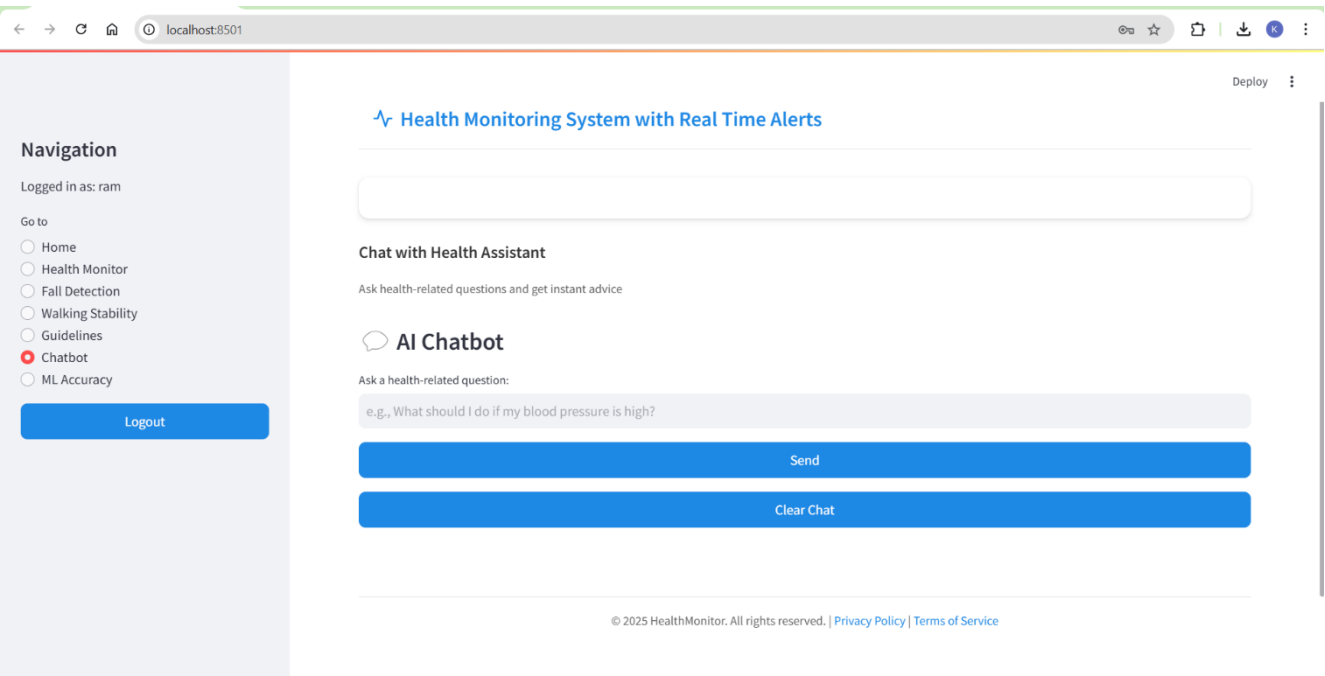


Figure 8.6 Chatbot

Guidelines:

- The Health Monitoring System with Real-Time Alerts guidelines page provides a virtual doctor explanation, step-by-step usage instructions, speech output for accessibility, and general health tips to improve user health and prevent falls.

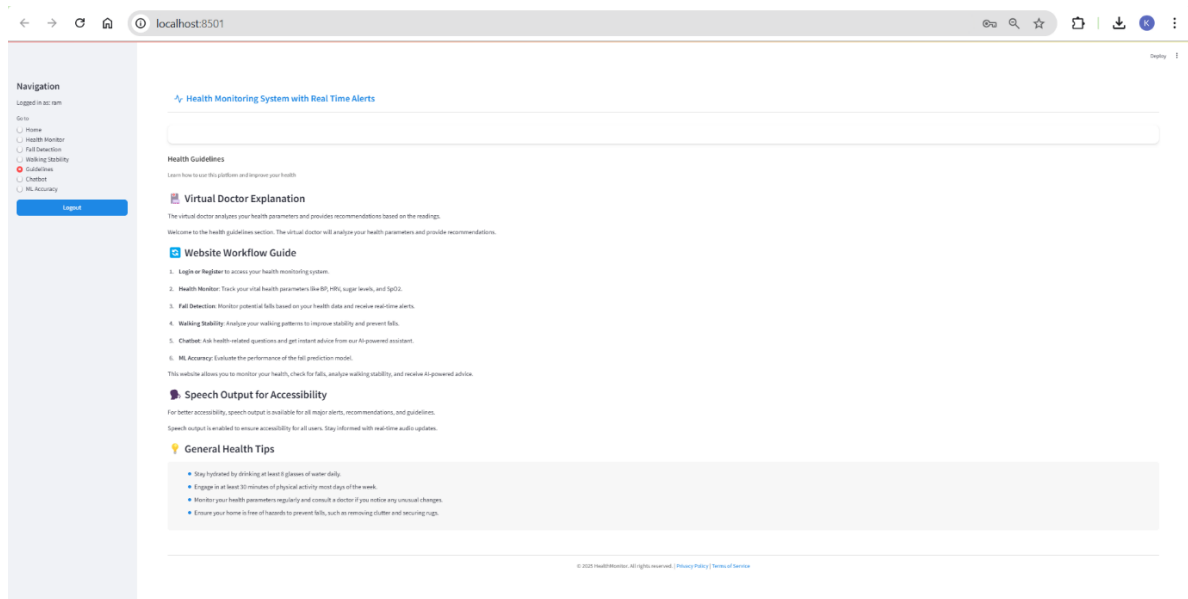


Figure 8.7 Guidelines

Walking Stability:

- A health monitoring system interface displays a step count image upload feature with a "Very Low" stability prediction and high fall risk alert.

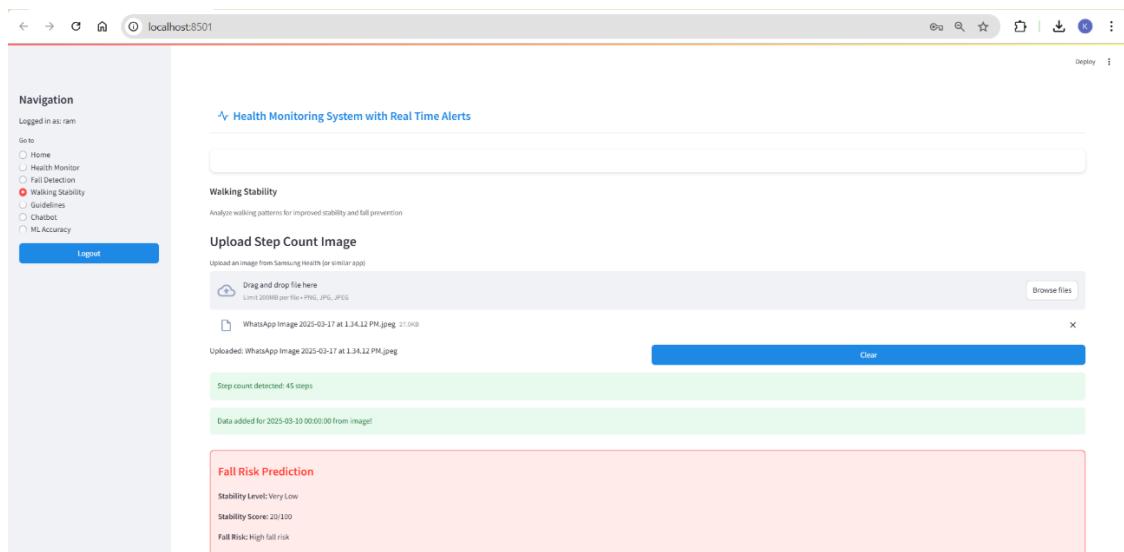


Figure 8.8 Walking Stability

A health monitoring system interface shows a stability meter with a score of 75, indicating "Good" walking steadiness on a color-coded scale from "Very Low" to "Very Good."



Figure 8.8.1 Walking Stability

A health monitoring system interface allows users to upload step count data and analyze walking steadiness, displaying a stability meter with a score of 75.

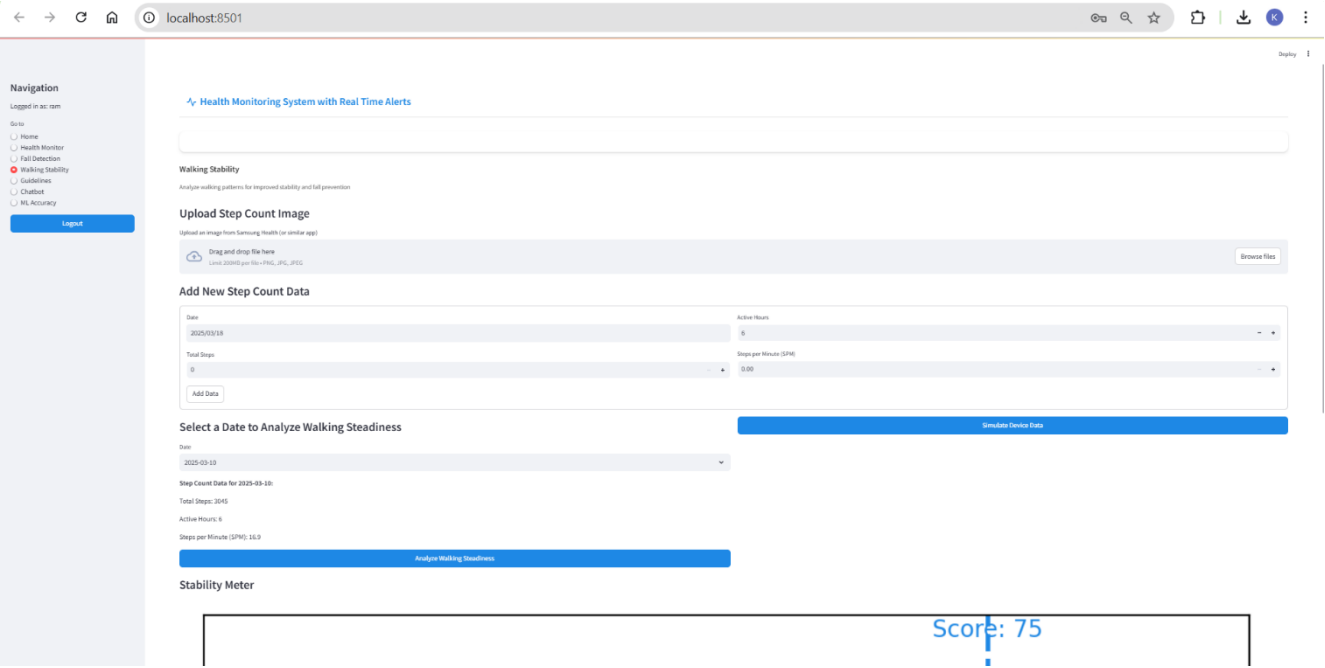


Figure 8.8.2 Walking Stability

ML Accuracy:

- The Health Monitoring System with Real-Time Alerts machine learning model accuracy page evaluates the performance of the fall prediction model, displaying a 100% accuracy for the SVM model and a dataset preview of health parameters.

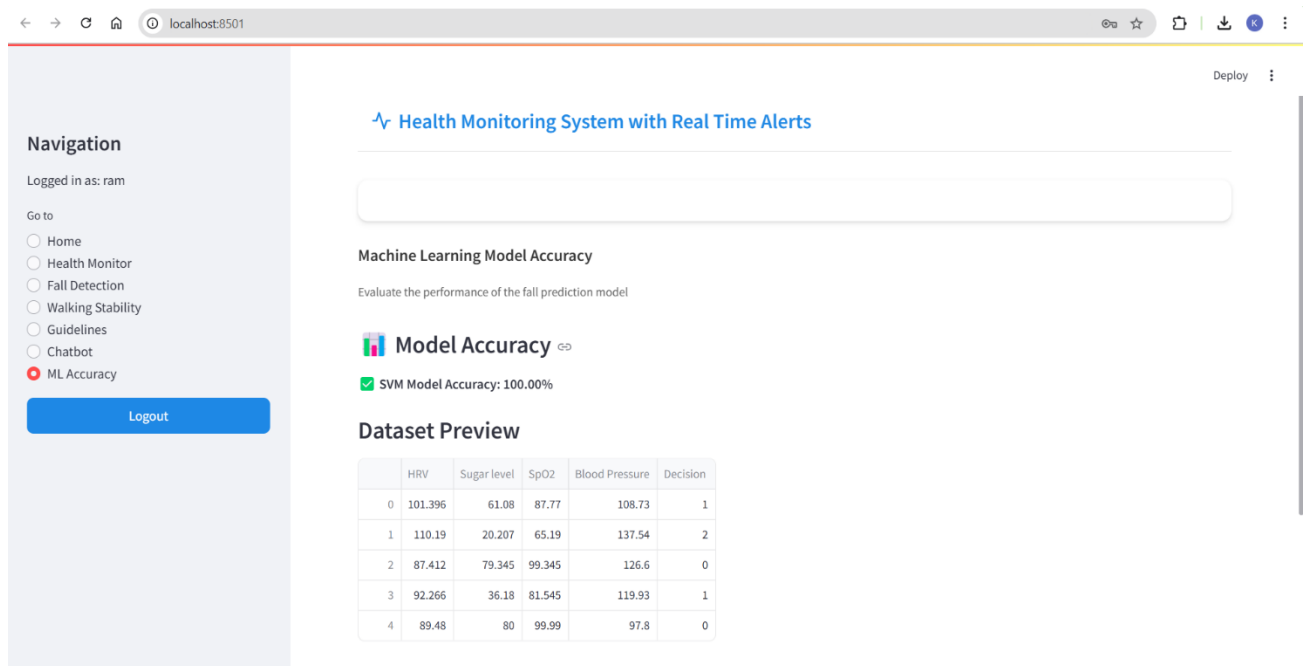


Figure 8.9 ML Accuracy

B – Sample Code

```
import streamlit as st
import pickle
import pyttsx3
import pandas as pd
import os
from twilio.rest import Client
import google.generativeai as genai
import numpy as np
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
# Load the dataset (same one used for training)
data_path = r"C:\Users\KEERTI\OneDrive\Documents\Desktop\fall
prediction\fall_prediction\cStick_final.csv"
data = pd.read_csv(data_path)
data.columns = data.columns.str.strip() # Clean column names
# Define Features and Target
X = data.drop(columns=['Decision'], errors='ignore')
y = data['Decision']
# Scale Features (Same Preprocessing as Training)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Train-Test Split X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)
# Load the trained model from the Pickle file
model_path = r"C:\Users\KEERTI\OneDrive\Documents\Desktop\fall
prediction\fall_prediction\svm_classifier.pkl"
def load_model():
    with open(model_path, "rb") as file:
        loaded_model = pickle.load(file)
    return loaded_model
def show_model_accuracy():
    model = load_model()
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    st.subheader(" Model Accuracy")
    st.write(f" **SVM Model Accuracy: {accuracy:.2%}**") # Display as percentage
# Set your Gemini API key
GEMINI_API_KEY = "AIzaSyAiWRuatBc3GlDdBkiRk7DwSZ6nPjbX8Js"
genai.configure(api_key=GEMINI_API_KEY)
# Function to generate chatbot responses
def chat_with_gemini(user_input):
```

```

try:
    model = genai.GenerativeModel("gemini-pro")
    response = model.generate_content(user_input)
    return response.text
except Exception as e:
    return f"Error: {e}"
# Streamlit UI for the chatbot
def chatbot():
    st.subheader(" AI Chatbot")
    user_input = st.text_input("Ask me anything:")
    if st.button("Send"):
        if user_input:
            response = chat_with_gemini(user_input)
            st.write(" AI Response:", response)
        else:
            st.warning("Please enter a message.")
# Twilio Credentials (Use environment variables for security)
TWILIO_ACCOUNT_SID = os.getenv("TWILIO_ACCOUNT_SID",
    "ACdc6f258e897aadb6c38e769124c2324c")
TWILIO_AUTH_TOKEN = os.getenv("TWILIO_AUTH_TOKEN",
    "37d7220577e73aaf75df345690af9cec")
TWILIO_PHONE_NUMBER = "+16282039907"
USER_PHONE_NUMBER = "+918074785887"
# Initialize Twilio Client
client = Client(TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN)
# Initialize Text-to-Speech engine
def speak(text):
    engine = pyttsx3.init()
    engine.say(text)
    engine.runAndWait()
    st.write(text)
# Function to detect fall based on health parameters
def detect_fall(row):
    try:
        bp = float(row["BP"]) if pd.notna(row["BP"]) else 0
        hrv = float(row["HRV"]) if pd.notna(row["HRV"]) else 0
        spo2 = float(row["SpO2"]) if pd.notna(row["SpO2"]) else 0
        if bp < 90 or hrv < 30 or spo2 < 85:
            return "Fall Detected"
        return "No Fall"
    except ValueError:
        return "Invalid Data" # Function to send SMS alert
def send_sms(message):
    try:

```



```

client.messages.create(
    body=message,
    from_=TWILIO_PHONE_NUMBER,
    to=USER_PHONE_NUMBER
)
st.write(f"[SMS Sent] {message}")
except Exception as e:
    st.write(f"Error sending SMS: {e}")
# Function to make call alert
def make_call():
    try:
        client.calls.create(
            twiml="Alert! A fall has been detected. Immediate attention is required.",
            from_=TWILIO_PHONE_NUMBER,
            to=USER_PHONE_NUMBER
        )
        st.write("[Call Alert] Call initiated successfully.")
    except Exception as e:
        st.write(f"Error making call: {e}")
# Define the CSV file for storing user data
USER_DATA_FILE = "patient_data.csv"
# Function to check if CSV file exists, if not create one
def initialize_user_data():
    if not os.path.exists(USER_DATA_FILE):
        df = pd.DataFrame(columns=["Username", "Password", "Age", "Blood
        Group", "Food Allergies"])
        df.to_csv(USER_DATA_FILE, index=False)
# Registration Function (Updated)
def register():
    st.subheader("Register New Patient")
    new_username = st.text_input("New Username")
    new_password = st.text_input("New Password", type="password")
    phone_number = st.text_input("Phone Number")
    age = st.number_input("Age", min_value=0, max_value=120, step=1)
    blood_group = st.selectbox("Blood Group", ["A+", "A-", "B+", "B-", "O+", "O-",
    "AB+", "AB-"])
    food_allergies = st.text_area("Food Allergies (comma-separated)")
    if st.button("Register"):
        if new_username and new_password and phone_number:
            if os.path.exists(USER_DATA_FILE):
                user_data = pd.read_csv(USER_DATA_FILE)
                if new_username in user_data['Username'].values:
                    st.warning("Username already exists. Please choose a different username.")
            return

```

```

else:
    user_data = pd.DataFrame(columns=["Username", "Password", "Phone
    Number", "Age", "Blood Group", "Food Allergies"])
    new_data = pd.DataFrame([[new_username, new_password, phone_number, age,
    blood_group, food_allergies]],
    columns=["Username", "Password", "Phone Number", "Age", "Blood
    Group", "Food Allergies"])
    user_data = pd.concat([user_data, new_data], ignore_index=True)
    user_data.to_csv(USER_DATA_FILE, index=False)
    st.success(f"User {new_username} registered successfully!")
    speak(f"User {new_username} registered successfully!")
else:
    st.warning("Please enter all details!")
# Login Function
def login():
    st.markdown(
        """
        <style>
        .top-right {
        position: absolute;
        top: -60px; /* Adjust the vertical position */
        right: -30px; /* Adjust the horizontal position */
        width: 285px; /* Adjust width */
        height: auto; /* Maintain aspect ratio */
        transform: scaleX(-1); /* Flip horizontally */
        }
        </style>
        
        """,
        unsafe_allow_html=True )
    # Image beside login (right-center)
    st.markdown(
        """
        <style>
        .right-center {
        position: absolute;
        top: 30%; /* Move to center vertically */
        right: -300px; /* Adjust horizontal position */
        transform: translateY(-30%); /* Perfectly center the image */
        width: 280px; /* Adjust width */
        height: 400px; /* Maintain aspect ratio */

```

```

}
</style>

""",
unsafe_allow_html=True
)
st.subheader("Login")
username = st.text_input("Username").strip()
password = st.text_input("Password", type="password").strip()
if st.button("Login"):
    if os.path.exists(USER_DATA_FILE):
        user_data = pd.read_csv(USER_DATA_FILE, dtype={'Password': str}) #
        force password to be string
        user_row = user_data[user_data["Username"].str.strip() == username]
        if not user_row.empty and user_row.iloc[0]["Password"].strip() == password:
            st.success(f"Logged in as {username}")
            speak(f"Logged in as {username}")
            st.session_state.logged_in = True
            st.session_state.username = username
        else:
            st.error("Invalid username or password.")
    else:
        st.error("No users registered. Please register first.")
# Default values for inputs
default_values = {"BP": 120.0, "HRV": 70.0, "Sugar Levels": 100.0, "SpO2": 95.0}
FALL_DETECTION_FILE = "fall_detection_data.csv"
# Initialize CSV file if not exists
def initialize_fall_data():
    if not os.path.exists(FALL_DETECTION_FILE):
        df = pd.DataFrame(columns=["Day", "BP", "HRV", "Sugar Levels", "SpO2",
            "Fall Status"])
        df.to_csv(FALL_DETECTION_FILE, index=False)
# Function to detect fall based on predefined conditions
def detect_fall(row):
    if row["BP"] < 90 or row["SpO2"] < 90:
        return "Fall Detected"
    return "No Fall"
# Fall prediction function
def fall_prediction():
    st.subheader("Fall Detection")
    days = [f"Day {i}" for i in range(1, 16)]
    data = pd.DataFrame(index=days, columns=["BP", "HRV", "Sugar Levels",

```

```

"SpO2"])
for day in days:
    col1, col2, col3, col4 = st.columns(4)
    # Use default values if no input is provided
    data.loc[day, "BP"] = col1.number_input(f"{day} BP", min_value=0.0,
    max_value=200.0, step=1.0, value=default_values["BP"])
    data.loc[day, "HRV"] = col2.number_input(f"{day} HRV", min_value=0.0,
    max_value=200.0, step=1.0, value=default_values["HRV"])
    data.loc[day, "Sugar Levels"] = col3.number_input(f"{day} Sugar", min_value=0.0,
    max_value=300.0, step=1.0, value=default_values["Sugar Levels"])
    data.loc[day, "SpO2"] = col4.number_input(f"{day} SpO2", min_value=50.0,
    max_value=100.0, step=1.0, value=default_values["SpO2"])
    if st.button("Check Fall Prediction"):
        data["Fall Status"] = data.apply(detect_fall, axis=1)
        if "Fall Detected" in data["Fall Status"].values:
            # Send alerts
            send_sms("ALERT: Fall detected! Immediate attention required.")
            make_call()
            speak("Fall Detected! Immediate attention required.")
            st.markdown("<p style='color:red; font-weight:bold;'>Fall Detected!
            </p>",unsafe_allow_html=True)
        else:
            speak("No fall detected. Stay safe!")
            st.success("No fall detected. Stay safe!")
            # Load existing fall detection data
            if os.path.exists(FALL_DETECTION_FILE):
                fall_data = pd.read_csv(FALL_DETECTION_FILE)
            else:
                fall_data = pd.DataFrame(columns=["Day", "BP", "HRV", "Sugar Levels", "SpO2",
                "Fall Status"])
            # Append new data
            data["Day"] = days # Add day column
            fall_data = pd.concat([fall_data, data], ignore_index=True)
            fall_data.to_csv(FALL_DETECTION_FILE, index=False)
            st.success("Fall detection data saved successfully!")
            # Initialize CSV before running Streamlit app
            initialize_fall_data()
            # Health Monitoring Function (Updated Parameters)
            def health_conditions():
                st.subheader(" Health Condition Monitoring")
                bp = st.number_input("Blood Pressure (BP)", min_value=50, max_value=200,
                value=120)
                hrv = st.number_input("Heart Rate Variability (HRV)", min_value=0,
                max_value=100, value=50)

```

```

sugar_level = st.number_input("Sugar Levels", min_value=0, max_value=300,
value=100)
spo2 = st.number_input("SpO2 Level", min_value=50, max_value=100,
value=95)
if st.button("Check Health Condition"):
    message = ""
    speech_message = ""
    # BP Check
    if bp < 90:
        message += " Warning! Low blood pressure detected.\n"
        speech_message += "Warning! Low blood pressure detected.\n Drink water, eat
something salty, and rest. If symptoms persist, consult a doctor. "
    elif bp > 140:
        message += " Warning! High blood pressure detected.\n"
        speech_message += "Warning! High blood pressure detected. \n Take your
prescribed medication, avoid salty foods, and relax. Consult a doctor if needed. "
    else:
        message += " Blood pressure is in normal range.\n"
        speech_message += "Blood pressure is in normal range. "
    # HRV Check
    if hrv < 20:
        message += " Warning! Low heart rate variability detected.\n"
        speech_message += "Warning! Low heart rate variability detected.\nTake a tablet
if prescribed and eat something nutritious. "
    else:
        message += " HRV levels are normal.\n"
        speech_message += "HRV levels are normal. "
    # Sugar Levels Check
    if sugar_level < 70:
        message += " Warning! Low blood sugar detected.\n"
        speech_message += "Warning! Low blood sugar detected. \n Take glucose or eat
something sugary immediately "
    elif sugar_level > 180:
        message += " Warning! High blood sugar detected.\n"
        speech_message += "Warning! High blood sugar detected. \n Take your
prescribed medication and follow a balanced diet "
    else:
        message += " Blood sugar is in normal range.\n"
        speech_message += "Blood sugar is in normal range. "
    # SpO2 Level Check
    if spo2 < 90:
        message += " Warning! Low oxygen levels detected.\n"
        speech_message += "Warning! Low oxygen levels detected.\n Take deep
breaths, rest, and consider using oxygen support if needed. "

```

```

else:
    message += " SpO2 levels are normal.\n"
    speech_message += "SpO2 levels are normal. "
    # Display warnings in UI
    st.write(message)
    # Speak all health conditions (both normal & warnings)
    speak(speech_message)
    # Health Guidelines Section
    def health_guidelines():
        st.subheader("Health Guidelines")
        # Virtual Doctor Explanation
        st.markdown("### Virtual Doctor Explanation")
        st.write("The virtual doctor will analyze your health parameters and provide recommendations based on the readings.")
        speak("Welcome to the health guidelines section. The virtual doctor will analyze your health parameters and provide recommendations.")
        # Website Workflow Guide
        st.markdown("### Website Workflow Guide")
        st.write("1. Login or Register to access your health monitoring system.")
        st.write("2. Check for Fall Detection to monitor potential falls based on your health data.")
        st.write("3. Monitor your health conditions and receive real-time alerts.")
        st.write("4. Follow the Health Guidelines for better health management.")
        speak("This website allows you to monitor your health, check for falls, and receive alerts based on your data.")
        # Speech Output for Accessibility
        st.markdown("### Speech Output for Accessibility")
        st.write("For better accessibility, speech output is available for all major alerts and guidelines.")
        speak("Speech output is enabled to ensure accessibility for all users. Stay informed with real-time audio updates.")
        # Streamlit App Layout
        st.markdown("<h1 style='align: center; color: black;'>Health Monitoring System with Real Time Alerts<h1>", unsafe_allow_html=True)
        # Initialize session state
        if 'logged_in' not in st.session_state:
            st.session_state.logged_in = False
        if 'username' not in st.session_state:
            st.session_state.username = None
        initialize_user_data() # Initialize user data
        if not st.session_state.logged_in:
            menu = st.sidebar.selectbox("Menu", ["Login", "Register"])
            if menu == "Login":
                login()

```

```
elif menu == "Register":
    register()
else:
    menu = st.sidebar.selectbox("Menu", ["Fall Detection", "Health Conditions",
    "Health Guidelines", "Chatbot", "ML Accuracy"])
    if menu == "Fall Detection":
        fall_prediction()
    elif menu == "Health Conditions":
        health_conditions()
    elif menu == "Health Guidelines":
        health_guidelines()
    elif menu == "Chatbot":
        chatbot()
    elif menu == "ML Accuracy":
        show_model_accuracy()
```